Parallel Space-Time Kernel Density Estimation

Erik Saule[†], Dinesh Panchananam[†], Alexander Hohl[‡], Wenwu Tang[‡], Eric Delmelle[‡]
[†] Dept. of Computer Science, [‡]Dept. of Geography and Earth Sciences
UNC Charlotte
Email: {*esaule,dpanchan,ahohl,wtang4,eric.delmelle*}*@uncc.edu*

August 19, 2018

Abstract

The exponential growth of available data has increased the need for interactive exploratory analysis. Dataset can no longer be understood through manual crawling and simple statistics. In Geographical Information Systems (GIS), the dataset is often composed of events localized in space and time; and visualizing such a dataset involves building a map of where the events occurred.

We focus in this paper on events that are localized among three dimensions (latitude, longitude, and time), and on computing the first step of the visualization pipeline, space-time kernel density estimation (STKDE), which is most computationally expensive. Starting from a gold standard implementation, we show how algorithm design and engineering, parallel decomposition, and scheduling can be applied to bring near real-time computing to space-time kernel density estimation. We validate our techniques on real world datasets extracted from infectious disease, social media, and ornithology.

1 Introduction

The rapid propagation of infectious diseases (e.g. Zika, Ebola, H1N1, Dengue fever) is conducive to serious, epidemic outbreaks, posing a threat to vulnerable populations. Such diseases have complex transmission cycles, and effective public health responses require the ability to monitor outbreaks in a timely manner [EE11]. Space-time statistics facilitate the discovery of disease dynamics including rate of spread, seasonal cyclic patterns, direction, intensity (i.e. clusters), and risk of diffusion to new regions. However, obtaining accurate results from space-time statistics is computationally very demanding, which is problematic when public health interventions are promptly needed. The issues of computational efforts are exacerbated with spatiotemporal datasets of increasing size, diversity and availability [GWM14]. High-performance computing reduces the effort required to identify these patterns, however heterogeneity in the data must be accounted for [HDTC16].

Epidemiology is only one of the application domains where it is important to understand how events occurring at different locations and different times form clusters. Political analysis, social media analysis, or the study of animal migration also require the understanding of spatial and temporal locality of events. The massive amount of data we see nowadays is often analyzed using complex models. But before these can be constructed, data scientists need to interactively visualize and explore the data to understand its structure.

In this paper, we present the space-time kernel density estimation application which essentially builds a 3D density map of events located in space and time. This problem is computationally expensive using existing algorithms. That is why we developed better sequential algorithms for this problem that reduced the complexity by orders of magnitude. And to bring the runtime in the realm of near real-time, we designed parallel strategies for shared memory machines.

Section 2 presents the STKDE application along with a reference implementation that uses a voxel-based algorithm VB to compute STKDE. The problem resembles the spatial summation of radial basis function, however the functions we consider are not radially symmetric.

In Section 3, we investigate the sequential problem and develop a point-based algorithm PB that significantly reduce the complexity compared to VB. We also identify invariants linked to the structure of the kernel density estimate

function that allows to reduce the computational cost by an additional factor and build the PB-SYM algorithm based on that observation. These invariants do not exist in the problem of summing radial basis functions which makes the problem we address have different properties and optimizations.

We develop a simple parallel strategy PB-SYM-DR in Section 4 which replicates the state of the problem, enables pleasingly parallel computation, and avoids a race conditions at the cost of performing a reduction at the end.

To reduce the cost of this reduction, we then investigate decomposing the space in blocks that can be independently processed. The PB-SYM-DD is built on that principle and is described in Section 5. It makes the the computation pleasingly parallel however it introduces overhead when the boundary of a block is too close to many events. We propose different methods to a space decomposition based on optimal hierarchical (Section 5.3.1) and jagged partitioning (Section 5.3.2), overdecomposition (Section 5.3.3) and heuristic partitioning based on block and octree decompositions (Section 5.4).

Since both previous strategies may introduce a significant work overhead (due to the reduction and the overhead of the decomposition), we propose PB-SYM-PD in Section 6 that provide a work-efficient decomposition of the points. However that algorithm has internal dependencies which can induce a long critical path and cause load imbalance. Alternative ordering of the work can reduce the length of the critical path. However finding the optimal ordering is difficult problem for which we present complexity, approximation results, and a heuristic using to build PB-SYM-PD-SCHED in Section 6.2.2. To reduce the critical path further, we propose the PB-SYM-PD-REP algorithm in Section 6.3 which introduces some work overhead only where is needed to achieve a shorter critical path.

Section 7 presents experimental results on 4 different datasets and 21 instances extracted from these datasets. The experiments highlights that the sequential strategies are efficient. The parallel strategies have their pros and cons and different strategies appear to be best in different scenarios.

Section 8 discusses the applicability of related problems and techniques.

A preliminary version of this paper has appeared in [SPH+17]. This paper extends the previous results by presenting a discussion of complex partitioning strategies for PB-SYM-DD (Sections 5.2, 5.3, and 5.4), the complexity and approximation result of work ordering for PB-SYM-PD (Section 6.2.2). This paper also presents more experimental results in Section 7.

2 Space-Time Kernel Density Estimation

2.1 Description

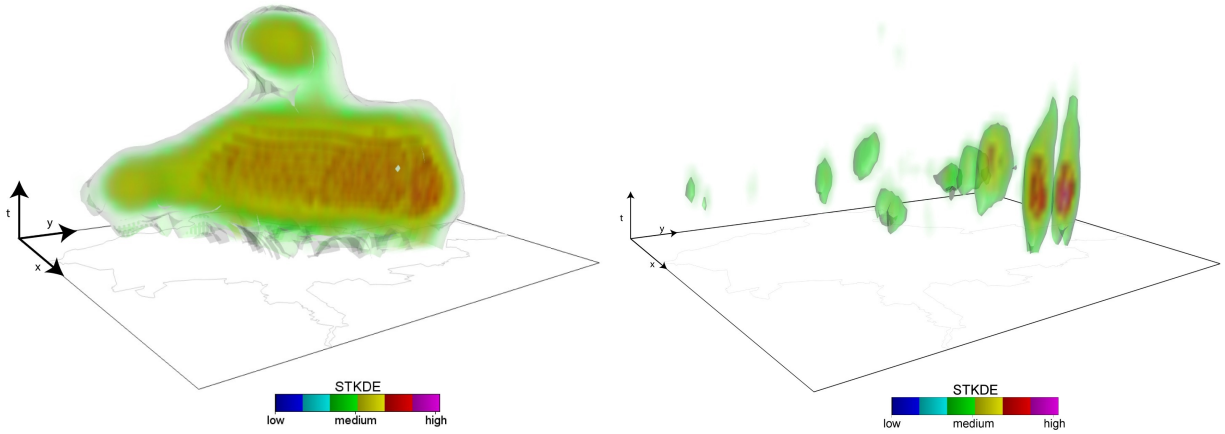
Space-time kernel density (STKDE) is used for identifying spatiotemporal patterns in datasets. It is a temporal extension of the traditional 2D kernel density estimation [Sil86] which generates density surface ("heatmap") from a set of n points located in a geographic space. The resulting density estimates are visualized within the space-time cube framework using two spatial (x, y) and a temporal dimension (t) [NY10]. STKDE creates a discretized 3D volume where each voxel (3D equivalent of a pixel) is assigned a density estimate based on the surrounding points. The space-time density is estimated using (following the notations of [HDTC16]):

$$\hat{f}(x,y,t) = \frac{1}{nh_s^2h_t} \sum_{i|d_i<h_s,t_i<h_t} k_s(\frac{x-x_i}{h_s}, \frac{y-y_i}{h_s})k_t(\frac{t-t_i}{h_t})$$

Density $\hat{f}(x, y, t)$ of each voxel is determined by number and distance of events (points) (x_i, y_i, t_i) within its vicinity, which is conceptualized by a cylinder. The spatial bandwidth h_s forms a circle which, due to the orthogonal relationship between space and time, is extended to a cylinder by temporal bandwidth h_t. For every event inside the cylinder, the spatial (d_i) and temporal (t_i) distances are smaller than h_s and h_t. Therefore, the event receives a weight based on the kernel functions k_s and k_t (distance decay):

$$k_s(u,v) = \frac{\pi}{2}(1-u)^2(1-v)^2$$

$$k_t(w) = \frac{3}{4}(1-w)^2$$



(a) $h_s = 2500m, h_t = 14days$ (b) $h_s = 500m, h_t = 7days$

Figure 1: Visualization of Dengue fever cases in Cali, Colombia in 2010 and 2011 for different spatial bandwidth and temporal bandwidth.

n	Number of points
$s = (x, y, t)$	A voxel and sampling coordinate
(x_i, y_i, t_i)	Coordinate of point i
h_s, h_t	Spatial and temporal bandwidth
g_x, g_y, g_t	Real size of the domain (in meters)
$sres, tres$	Resolution (in meters)
$s = (X, Y, T)$	A voxel in voxel space
(X_i, Y_i, T_i)	Voxel of point i
G_x, G_y, G_t	Size of the domain (in voxels)
H_s, H_t	Bandwidth (in voxels)

Table 1: Notations

Figure 1 illustrates how varying the bandwidths used in STKDE helps focusing the graphical visualization of Dengue fever cases in Cali, Colombia [DDC+14]. Figure 2 shows the impact of a single point on the neighboring space.

Computationally, the domain of size g_x, g_y, g_t is discretized in voxels using a spatial resolution $sres$ and a temporal resolution $tres$. Therefore, the domain is represented by a grid of size $G_x = \lceil \frac{g_x}{sres} \rceil$, $G_y = \lceil \frac{g_y}{sres} \rceil$, $G_t = \lceil \frac{g_t}{tres} \rceil$. Each point causes a density increase in the voxels that are within a cylinder centered on the point, of radius equal to the spatial bandwidth in voxels $H_s = \lceil \frac{h_s}{sres} \rceil$, and of half height equal to the temporal bandwidth $H_t = \lceil \frac{h_t}{tres} \rceil$.

All the notations are summarized in Table 1. As a convention all uppercase notations denote quantities in voxels and all lowercase notations denote quantities in domain space.

2.2 Gold Standard Implementation

The gold standard implementation (for instance from [HDTC16]) follows the exact definition of STKDE as it is given above. It is a voxel-based algorithm we call VB. For each voxel, VB finds the points within the temporal and spatial bandwidths and calculates the contribution to the density of this voxel. The pseudo code is given in Algorithm 1.

The algorithm performs $\theta(G_x G_y G_t n)$ distance tests and computes $\theta(n H_s^2 H_t)$ densities. Since H_s is smaller than G_x and G_y and since H_t is smaller than G_t, the complexity of the algorithm is $\theta(G_x G_y G_t n)$ and it requires $\theta(G_x G_y G_t)$ memory.

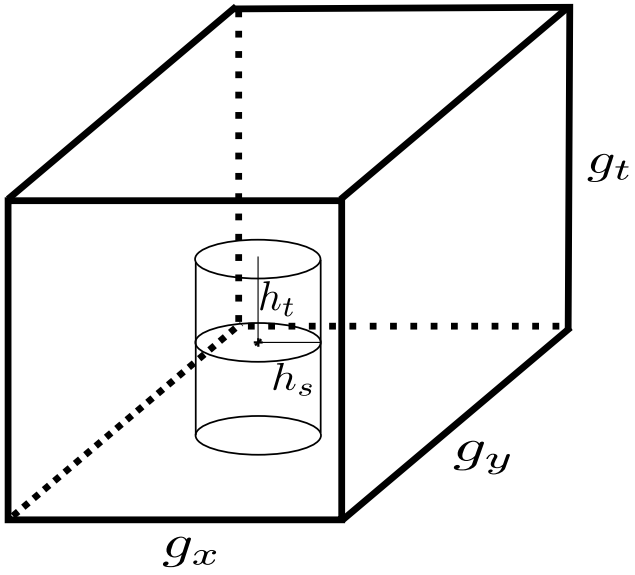


Figure 2: The computation of STKDE happens in a domain space of size g_x, g_y, g_t. Each point impacts the neighboring space at a distance h_s in space and h_t in time; forming a cylinder of diameter $2h_s$ and of height $2h_t$.

Algorithm 1 VB

for all voxels $s = (x, y, t)$ **do**
  $sum = 0$
  for all points i at x_i, y_i, t_i **do**
    if $\sqrt{(x_i - x)^2 + (y_i - y)^2} < h_s$ and $|t_i - t| \leq h_t$ **then**
      $sum+ = k_s(\frac{x-x_i}{h_s}, \frac{y-y_i}{h_s})k_t(\frac{t-t_i}{h_t})$
  $stkde[X][Y][T] = \frac{sum}{nh_s^2h_t}$

3 Algorithm Design and Engineering

3.1 Point-based Algorithm

The voxel-based algorithm suffers from a massive cost of computing distances. The gold implementation reduces the number of distance computation by decomposing the domain, but that still incurs many distance calculations. Most of these calculations are unnecessary since we know where each point will radiate density. The point-based algorithm PB, given in Algorithm 2, leverages that property.

Algorithm 2 PB

for all voxels $s = (x, y, t)$ **do**
  $stkde[X][Y][T] = 0$
for each points i at x_i, y_i, t_i **do**
  for $X_i - H_s \leq X \leq X_i + H_s$ **do**
    for $Y_i - H_s \leq Y \leq Y_i + H_s$ **do**
      for $T_i - T_s \leq T \leq T_i + H_s$ **do**
        if $\sqrt{(x_i - x)^2 + (y_i - y)^2} < h_s$ and $|t_i - t| \leq h_t$ **then**
          $stkde[X][Y][T] += \frac{k_s(\frac{x-x_i}{h_s}, \frac{y-y_i}{h_s}) k_t(\frac{t-t_i}{h_t})}{n h_s^2 h_t}$

The algorithm PB incurs mostly two costs. Initializing the memory which costs $\Theta(G_x G_y G_t)$ and computing the impact of each point $\Theta(n H_s^2 H_t)$. Note that it is possible that either of these two terms is significantly larger than the other. Therefore the complexity of the algorithm is $\Theta(G_x G_y G_t + n H_s^2 H_t)$.

3.2 Exploiting Symmetries

When the bandwidth is large, computing the density contribution of each point each for voxel in the bandwidth is the most expensive operation. Each of these $n(2H_s)^2 2H_t$ calculations costs approximately 40 floating point operations. (The exact cost is difficult to estimate since there are floating point additions, multiplications, divisions, and square root operations).

One can remark that the calculation is mostly redundant since for one particular point, its contribution to the neighboring voxels can be decomposed in two components. $K_s(x, y)$ only depends on the spatial coordinate of the voxel with respect to the point and does not depend on its temporal coordinate. And $K_t(t)$ only depends on the temporal coordinate and not on the spatial coordinates. See Figure 3 for a graphical depiction.

Therefore, we write an algorithm to leverage these symmetries in the problem. We present three variants, the PB-DISK variant ensures that the invariant on the spatial domain is only computed once. The PB-BAR variant ensures that the invariant on the temporal domain is only computed once. The PB-SYM variant computes independently the spatial invariant and the temporal invariant and then adds the density caused by the point by taking the product of the two invariants. (See pseudocode in Algorithm 3.) Notice that leveraging the symmetries in the calculation does not change the complexity (PB-SYM still has a complexity of $\Theta(G_x G_y G_t + n H_s^2 H_t)$) but reduces the number of flops.

Notice that this exploitation of symmetries is not possible in a voxel-based algorithm.

4 Domain Replication

The simplest way to parallelize this kind of computation is to split the points equally among the P different computational units. Though, if two close-by points are computed simultaneously, the cylinders of density around them might intersect and there is a race condition.

The PB-SYM-DR algorithm solves the data race issue by having each of the P computational unit aggregate the result on a local copy of the domain. Once all the points have been processed, the P copies need to be summed. The pseudo code of PB-SYM-DR is given in Algorithm 4.

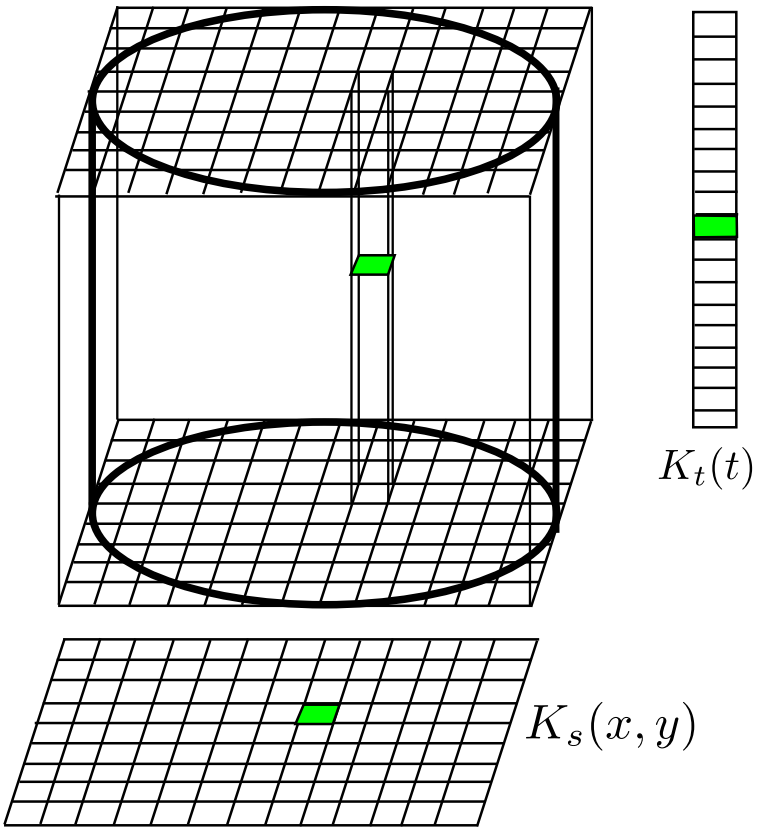


Figure 3: The contribution to the density estimate of each voxel of a cylinder can be decomposed in two terms, one temporally invariant $K_s(x, y)$ and one spatially invariant $K_t(t)$.

Algorithm 3 PB-SYM

for all voxels $s = (x, y, t)$ **do**
  $stkde[X][Y][T] = 0$
for each points i at x_i, y_i, t_i **do**
  for $X_i - H_s \leq X \leq X_i + H_s$ **do**
    for $Y_i - H_s \leq Y \leq Y_i + H_s$ **do**
      if $\sqrt{(x_i - x)^2 + (y_i - y)^2} < h_s$ **then**
        $K_s[X][Y] = \frac{k_s(\frac{x-x_i}{h_s}, \frac{y-y_i}{h_s})}{nh_s^2 h_t}$
      else
        $K_s[X][Y] = 0$
  for $T_i - T_s \leq T \leq T_i + H_s$ **do**
    if $|t_i - t| \leq h_t$ **then**
      $K_t[T] = k_t(\frac{t-t_i}{h_t})$
    else
      $K_t[T] = 0$
  for $X_i - H_s \leq X \leq X_i + H_s$ **do**
    for $Y_i - H_s \leq Y \leq Y_i + H_s$ **do**
      for $T_i - T_s \leq T \leq T_i + H_s$ **do**
        $stkde[X][Y][T] += K_s[X][Y]K_t[T]$

This algorithm has a memory requirement of $\Theta(PG_xG_yG_t)$ and a parallel amount of work of $\Theta(PG_xG_yG_t + nH_s^2H_t)$. Fortunately, the increase in work enables the computation to be pleasingly parallel. (There are actually in three pleasingly parallel phases: initializing the memory, processing the points, and reducing the partial results.)

Algorithm 4 PB-SYM-DR

for all processor $p \leq P$ in parallel **do**
  for all voxels $s = (x, y, t)$ **do**
    $stkdel[p][X][Y][T] = 0$
for each points i at x_i, y_i, t_i distributed among the P processors **do**
  Processor p processes i using PB-SYM and aggregate it in $stkdel[p]$
for all voxels $s = (x, y, t)$ in parallel **do**
  $stkde[X][Y][T] = 0$
  for all processor $p \leq P$ **do**
    $stkde[X][Y][T] += stkdel[p][X][Y][T]$

5 Domain Decomposition

5.1 Principles

Another strategy to avoid the data race is to compute voxel intensity independently for different subdomains. We call this method PB-SYM-DD (Algorithm 5). It splits the domain in AxBxC subdomains and each subdomain is associated the points which cylinder intersects with a voxel of the subdomain. Then the algorithm proceeds with handling each subdomain independently by only considering the impact of the data-point on voxels within the subdomain.

Algorithm 5 PB-SYM-DD

for all points $i = (x_i, y_i, t_i)$ **do**
  for each subdomain (a, b, c) **do**
    if $(X, Y, T) \pm (H_s, H_s, H_T)$ intersects
$(\lfloor \frac{aG_x}{A} \rfloor : \lceil \frac{(a+1)G_x}{A} \rceil, \lfloor \frac{bG_y}{B} \rfloor : \lceil \frac{(b+1)G_y}{B} \rceil, \lfloor \frac{cG_t}{C} \rfloor : \lceil \frac{(c+1)G_t}{C} \rceil)$ **then**
      $localpoints[a][b][c].add(x_i, y_i, t_i)$
for each subdomain (a, b, c) in parallel **do**
  Process subdomain (a, b, c) using PB-SYM

The main problem with Domain Decomposition is that it requires some points to be replicated across multiple subdomain which incurs additional work. Indeed, if a point is replicated in multiple subdomains, its cylinder is split across multiple subdomains as well. Assuming the split is temporal, then using PB-SYM in each subdomain requires computing part of the temporal invariant, the entire spatial invariant, and part of the cylinder. Therefore both subdomains need to compute the entire spatial invariant. See Figure 4 for a graphical depiction of this phenomenon.

If the algorithms keeps the number of subdomains small such that each point is replicated less than a constant number of times, then the work expressed in Big-Oh notation does not change. (This can be achieved by having subdomains larger than $H_s \times H_s \times H_t$.) However, the practical amount of work can increase by a non-negligable factor.

Another issue with Domain Decomposition is load imbalance. The points are unlikely to be equally distributed in the domain space, but more likely clustered around some locations. The subdomain containing a cluster will have a significantly higher work than other subdomains. And this work is not executed in parallel. One could decompose the domain more, but replicated point will incur an even higher overhead which may end up nullifying the gain in load balance.

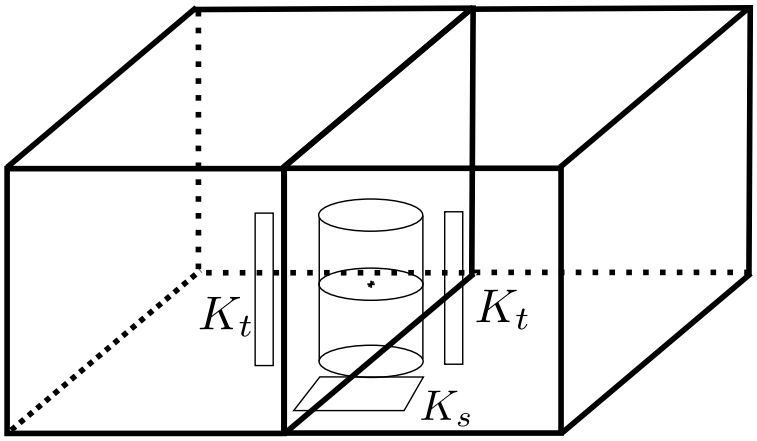


Figure 4: When a cylinder is split among two subdomains, PB-SYM-DD causes an overhead because one of the invariant needs to be recomputed. Here each subdomains compute part of the temporal invariant K_s, but both need to compute the spatial invariant K_t.

There must exist decomposition that are provided better compromise between available parallelism and overhead. First to be able to find better decomposition, we need to have a model for the load of a box and we provide a model in Section 5.2. Based on this model we investigate the problem of finding an optimal decomposition in Section and because the problem is NP-Complete in general, we consider relatively general decomposition patterns for which we can find an optimal decomposition, namely Hierarchical decomposition in Section 5.3.1 and Jagged decomposition in Section 5.3.2. Finally we consider faster heuristic decomposition in Section 5.4.

5.2 Work Inefficient Model

The first thing needed for obtaining better decomposition is a model of the load for regions of the space so that we can evaluate the quality of the decomposition

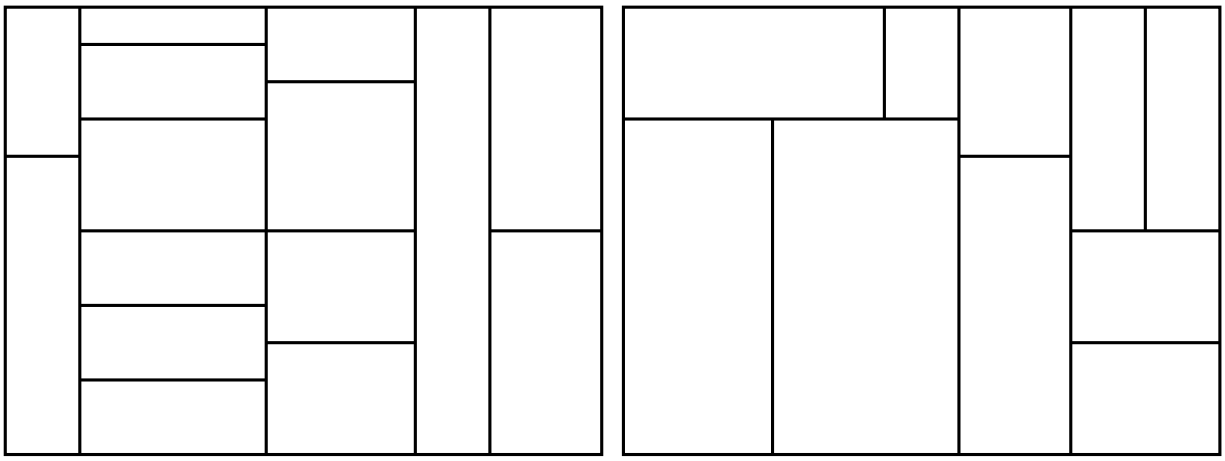
We model simply the work introduced by an observation obs on a voxel box vb as $Load(obs, vb)$. This function has three components, one represents the computation of k_s, one represents the computation of k_t, and the third represents the computation of the stkde array. $Load(obs, vb) = \alpha*((Proj_{XY}(Cyl(obs)) \bigcap vb)+\beta*((Proj_T(Cyl(obs)) \bigcap vb)+ \gamma * (Cyl(obs) \bigcap vb)$. There are three parameters α, β, and γ which represents the architectural constant of the system. These constant will be estimated using a micro benchmark.

The load of a box $Load(vb)$ is simply the sum of work introduced by all the observation that impact that box.

5.3 Exact Decomposition

Once we have a model of the load of a particular set of voxels, decomposing the load becomes a combinatorial problem. It turns out that spatial decomposition have different complexities depending of the structure of decomposition one is looking for. Even for a problems where the load is work efficient, finding the optimal spatial decomposition of arbitrary structure is an NP-Hard problem [GM96]. Rectilinear partitionings, which are checker board decomposition of varying square size, are also NP-Hard to optimize [GM96, AHM98].

However, other decomposition, such as jagged partitioning or any recursive partitioning based on boxes can be found optimally when the decomposition is work efficient [SBC12]. 2D variants of such decompositions can be seen on Figure 5. We show in this section how to discover optimal hierarchical partitioning and jagged partitioning when the decomposition is work inefficient.



(a) Jagged Partitioning (b) Hierarchical Decomposition

Figure 5: 2D representation of the two types of decomposition addressed in this paper. Jagged partitioning form vertical stripes which are then partitioned horizontally. Hierarchical decomposition are defined recursively by cutting the domain in a grid aligned manner.

5.3.1 Hierarchical Decomposition

By Hierarchical Decomposition, we actually mean generalized forms of Recursive Bisection [BB87]. That is to say, domains are recursively decomposed by applying a single grid aligned cut. Here we are looking at the 3D problem. So each box will be cut either by a grid aligned XY plane, or a grid aligned XT plane, or a grid aligned YT plane.

This structure makes the problem particularly suited for Dynamic Programming. Indeed, a particular sub-domain can be decomposed independently from any other boxes, as long as the number of processors used to decomposed a box is known in advance. This leads to the dynamic programming presented in Figure 6 for partitioning the box $X_{min}, X_{max}, Y_{min}, Y_{max}, T_{min}, T_{max}$ for P processors.

This Dynamic Programming algorithm computes the optimal solution by exploring $O(G_x^2 G_y^2 G_t^2 P)$ states; each intermediate state being computed in $O((G_x + G_y + G_t)P)$, and each final state being an evaluation of the $Load$ function. Clearly this complexity is too high for the decomposition to be computed in a reasonable amount of time. In practice, one can obtain a reasonable approximation of the optimal solution by only authorizing cuts on voxel which X coordinate is a multiple of a parameter $Xstep$. Doing so reduces the number of state by a factor of $Xstep^2$. Similar parameters $Ystep$ and $Tstep$ can be introduced.

Implementing this Dynamic Programming algorithm is fairly straightforward. Not all the state will be reached in practice so it is better to implement the function the dynamic programming top down (recursively) with memoization; rather than bottom-up. This implementation becomes faster when realizing a few properties of the formulation. For instance, when evaluation the best x position to cut the domain, if one already has a valid decomposition of the box and if the first term $Hier(X_{min}, x, Y_{min}, Y_{max}, T_{min}, T_{max}, p)$ is higher than the value of the currently best known decomposition, one does not need to evaluate the best decomposition for the second term $Hier(x, X_{max}, Y_{min}, Y_{max}, T_{min}, T_{max}, P - p)$. Even better, the first term will only increase when x increases and the second term will only decrease when x increases. So if a particular value of x sees its first term leading to a sub-decomposition worst that the currently best known decomposition, then all subsequent values of x which are all larger, will only lead to decomposition that won't be picked in practice. So the corresponding states of the dynamic programming do not need to be evaluated.

$$
\begin{aligned}
Hier(X_{min}, X_{max}, Y_{min}, Y_{max}, T_{min}, T_{max}, P) = \min_{1 \le p < P} (& \\
& \min_{X_{min} < x < X_{max}} (\max Hier(X_{min}, x, Y_{min}, Y_{max}, T_{min}, T_{max}, p), \\
& \qquad Hier(x, X_{max}, Y_{min}, Y_{max}, T_{min}, T_{max}, P - p)), \\
& \min_{Y_{min} < y < Y_{max}} (\max Hier(X_{min}, X_{max}, Y_{min}, y, T_{min}, T_{max}, p), \\
& \qquad Hier(X_{min}, X_{max}, y, Y_{max}, T_{min}, T_{max}, P - p)), \\
& \min_{T_{min} < t < T_{max}} (\max Hier(X_{min}, X_{max}, Y_{min}, Y_{max}, T_{min}, t, p), \\
& \qquad Hier(X_{min}, X_{max}, Y_{min}, Y_{max}, t, T_{max}, P - p)))
\end{aligned} \tag{1}
$$

$$
Hier(X_{min}, X_{max}, Y_{min}, Y_{max}, T_{min}, T_{max}, 1) = Load(X_{min}, X_{max}, Y_{min}, Y_{max}, T_{min}, T_{max}) \tag{2}
$$

$$
Hier(X_{min}, X_{min}{+}1, Y_{min}, Y_{min}{+}1, T_{min}, T_{min}{+}1, P) = Load(X_{min}, X_{min}{+}1, Y_{min}, Y_{min}{+}1, T_{min}, T_{min}{+}1) \tag{3}
$$

Figure 6: Dynamic Programming formulation of the Hierarchical Partitioning method.

5.3.2 Jagged Decomposition

Jagged decompositions are a simpler pattern than hierarchical recursive decomposition. As such, they are expected to lead to simpler structures, but they should be faster to compute. A jagged decomposition first stripes the domain along the X dimension. Then each stripe is individually striped along the Y dimension. The remaining space is striped along the T dimension. This simpler pattern still allows many decompositions, but limits the combinatorial explosion compared to hierarchical patterns.

In a work-efficient decomposition and in the one dimensional case, one can easily build an index of of the load which allows to quickly evaluate the cost of a particular sub-domain; and in particular, it is easy to chose the size of any sub-domain to expand up to the predicted optimal value of the decomposition. This was first used by Nicol [Nic94] and the technique was refined by Pinar [PA04]. This was extended to deal with heterogeneous processors in [PTA08] and to handle jagged 2D decompositions in [SBC12]

Unfortunately in a work inefficient problem, there is no easy indexing that can be built and target value for the solution of the problem can not easily be leveraged to make similar types of optimization. This force us to use a more complex form of the problem which relies on a recursive dynamic programming formulation. At the highest level, the domain will be split along the X dimension at a particular location. The right part will processed with a jagged partitioning along the Y and Z axes. The left part will recursively be processed with a jagged partitioning along the X, Y, and Z axes. For each cut distributing the processors on either sides is necessary. The second level has a similar structure except the cuts will be along the Y axis and deeper partitioning will be along the Z axis. At the level of the Z cuts, the only difference is that there is no need to distribute the processors on either side of the cut, since one side will always take one processor.

The complete formulation of the dynamic programming is provided in Figure 7. Most of the cost of the formulation comes from the evaluation of all the possible $JaggedT$ states: there are $O(G_x^2 G_y^2 G_t P)$ of them and each cost $O(G_t)$ calls to $Load$ to evaluate.

This dynamic programming formulation also has similar optimization as the hierarchical dynamic programming allowing not to evaluate all the state based on current knowledge of what the best local solution is. It can also be made faster by limiting the decomposition to values of x, y, and t which are multiples of parameters $Xstep$, $Ystep$, and $Tstep$.

$$JaggedX(X_{max}, P) = \min JaggedY(0, X_{max}, G_y, P), \min_{0<p<P} \min_{0<x<X_{max}} \max JaggedX(x, p), JaggedY(x, X_{max}, G_y, P-p) \quad (4)$$

$$JaggedY(X_{min}, X_{max}, Y_{max}, P) = \min JaggedT(X_{min}, X_{max}, 0, Y_{max}, G_t, P), \min_{0<p<P} \min_{0<y<Y_{max}} \max JaggedY(X_{min}, X_{max}, y, p), JaggedT(X_{min}, X_{max}, y, Y_{max}, G_t, P-p) \quad (5)$$

$$JaggedT(X_{min}, X_{max}, Y_{min}, Y_{max}, T_{max}, P) = \min_{0<t<T_{max}} \max JaggedT(X_{min}, X_{max}, Y_{min}, Y_{max}, t, P-1), Load(X_{min}, X_{max}, Y_{min}, Y_{max}, t, T_{max}) \quad (6)$$

$$JaggedT(X_{min}, X_{max}, Y_{min}, Y_{max}, T_{max}, 1) = Load(X_{min}, X_{max}, Y_{min}, Y_{max}, 0, T_{max}) \quad (7)$$

Figure 7: Dynamic programming formulation of the Jagged Partitioning

5.3.3 Overdecomposition

Despite having optimal exact decomposition, this may not lead to the best parallel performance in many practical cases. First obtaining the optimal decomposition assumed that one can have a perfect model of the execution of an application on a parallel platform. In practice, this can be really difficult as estimating the parameters of models accurately is difficult; in particular because lots of interference can make the model inaccurate. Also it is possible to find the best partitioning in, say, 16 sub-domains. But it is possible that there is no good decomposition for this particular number of processors.

A simple solution to solve the over-decomposition problem would eb to run the previously mentioned algorithms using number of sub-domains. However, in a work inefficient application, this could lead to a significant overhead. Indeed, these dynamic programming methods aim at minimizing the size of the largest sub-domain. However, in an over decomposition this may not be the only concern.

The concern when over decomposing is to achieve an execution that is fast, but the size of a particular sub-domain may no longer be that relevant. It is difficult to predict precisely how an execution with many sub-domains will unfold. And even if we could, that formulation is unlikely to be easy to optimize.

However, one can simply model the execution of sub-domains as independent tasks. Scheduling independent tasks has a wide literature [Leu04], but really early works of Graham [Gra66] are the most insightful for us. Graham showed that any greedy schedule of n tasks of processing time p_i on P processors will complete before $C_{max} \leq \frac{\sum p_i}{P} + \max p_i$. The two components of this formula are easily interpreted in terms of spatial decomposition, $p_{max} = \max p_i$ is the cost of the largest sub-domain and $\sum p_i$ is the total amount of work, including the overhead induced by the decomposition.

The optimization problem of interest is then of minimizing the largest sub-domain while minimizing the total amount of work (linearly related to the overhead). When optimizing for a multi-objective problem, all the best trade-off solution, known as the Pareto Set, can be of interest. However that set can be exponentially large. Fortunately, it is possible to build a good approximation of this Pareto Set by solving a set of problems where one of the objective is constrained to be smaller than a parameter while the other parameter is optimized [PY00]. Here we will chose to put a constraint on the largest sub-domain $p_{max} = \max p_i$ and minimize the total amount of work $\sum p_i$.

It turns out that doing this new optimization problem can be solved by adapting the dynamic programming algo-

rithms of the previous section. The cost of the solution implied by a cut in a domain becomes the sum of the of the cost of each side (instead of being the max). And the cost of an undivided sub-domain is still given by $Load$, except if that value is larger than the allowed cost of the sub-domain, in which case, the cost of that sub-domain is set to infinity.

5.4 Octree Decompositions

The methods that we presented before are either trivial heuristics (such as regular block decompositions) or of high complexity (such as optimal recursive decompositions). These methods are likely to either lead to bad quality partitions or to be too expensive to compute in practical scenarios.

We propose to use a classic idea to have an cheap yet adaptive decomposition inspired by octree decomposition of the space. The decomposition algorithm is initialized by having a single cuboid that covers the entire domain space. Then, iteratively, the cuboid of largest load (not necessarily of largest volume) is split in eight cuboid of identical volume. The process stops whenever the cuboid of largest load can not be divided, or a given number of cuboid is reached.

These octree decompositions are inherently adaptive. They are unlikely to over decompose the parts of the domain that have low load. They are likely good at ensuring that the cuboid of highest load has a fairly low load. However this approach does not take the work inefficiency of the decomposition into account to make the decision. So, dense region of the domain could be naively partitioned, leading to a potential high work overhead.

6 Point Decomposition

6.1 Principles

Both of the previous strategies increase the amount of work significantly. We want to achieve parallelism without cutting a cylinder and without requiring each thread to have its own memory space. To achieve that goal, we propose the PB-SYM-PD algorithm that decomposes the points in sets that can be safely performed simultaneously.

As long as two points are separated by $2h_s$ in space dimension or by $2h_t$ in time, the cylinders induced by the two points will not overlap. PB-SYM-PD decomposes the points in $A \times B \times C$ subdomain of identical size. As long as these subdomains are larger than $2h_s$ in a spatial dimension and $2h_t$ in the temporal dimension, the points in domain (x, y, z) can be done concurently with the points in $(x + 2, y, z)$. See Figure 8 for a graphical depiction.

We implemented the first version of this algorithm that organizes the subdomain in 8 sets where the first set is composed of the subdomains $(2i, 2j, 2k)$, the second of all the subdomains $(2i + 1, 2j, 2k)$, the third of all the subdomains $(2i, 2j + 1, 2k)$, etc.. The algorithm processes the sets the one after the other by doing each subdomain within a set in parallel. Two such sets are depicted in Figure 8. In practice, this is implemented using 8 OpenMP parallel-for constructs.

Algorithm 6 PB-SYM-PD

for all points $i = (x_i, y_i, t_i)$ **do**
  $localpoints[\lfloor \frac{AX}{G_x} \rfloor][\lfloor \frac{BY}{G_y} \rfloor][\lfloor \frac{CT}{G_t} \rfloor].add(x_i, y_i, t_i)$
for $abase \in \{0; 1\}$ **do**
  for $bbase \in \{0; 1\}$ **do**
    for $cbase \in \{0; 1\}$ **do**
      Do the work of the next three loops in parallel
      for $a = abase; a \leq A; a+ = 2$ **do**
        for $b = bbase; b \leq B; b+ = 2$ **do**
          for $c = cbase; c \leq C; c+ = 2$ **do**
            Process points in $localpoints[a][b][c]$ using PB-SYM

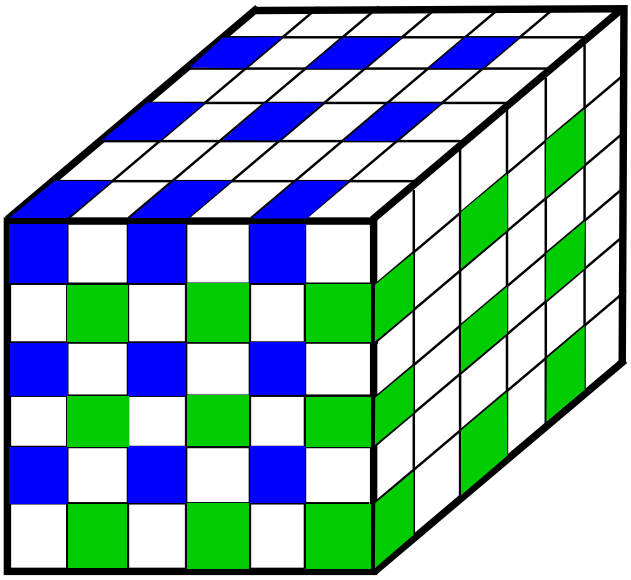


Figure 8: Provided the subdomains are larger than twice the bandwidth, the points contained in each blue boxes can be performed simultaneously.

6.2 Coloring and Scheduling based Method

6.2.1 Modeling as a Graph Coloring Problem

Note that such an implementation overconstrains the execution of the algorithm. Indeed, despite they are not in the same set, subdomain $(1, 0, 0)$ and $(64, 64, 64)$ can be computed at the same time. The real constraint is that the points contained in a subdomain can be processed safely as long as none of the point contained in a neighboring subdomain is being processed. Using the OpenMP tasking construct with dependency introduced in OpenMP 4.0 [Boa13], one can precisely express this constraint, which is a 27-point stencil constraint.

Formally speaking, this problem is a combination of multiple very classical graphs and scheduling problem. Modeling the subdomains as a 27-point stencil graph, identifying sets of subdomains to perform in parallel is coloring the vertices of the graph so that no two neighboring vertices share the same color. It differs from classical graph coloring problems (surveyed in [GMP05]) in that the objective is not to minimize the number of colors or to balance the number of vertices of each color but to minimize the execution time of the implied schedule.

The implied schedule is actually a scheduling of a graph of dependency which is extracted from the coloring. Each edge of the stencil graph is oriented from the vertex of the lowest color to the vertex with the highest color. Each vertex is associated with a processing time proportional to the number of points inside the sub-domain the vertex represents and the neighboring subdomains. This construction is represented in Figure 9.

The OpenMP scheduler is not explicit in which order tasks that are available are performed. Though it is reasonable to assume that the tasks will be scheduled using a greedy algorithm. Therefore, the classic guarantee from Graham's List Scheduling [Gra69] is valid:

$$T_P \leq \frac{T_1 - T_\infty}{P} + T_\infty,$$

where T_1 is the total amount of work to perform and T_∞ is the length of the longest chain in the dependency graph.

6.2.2 Interval Coloring of a 27-point Stencil Graph

We propose to reduce the critical path by using a load-aware coloring of the subdomains. Let us consider this problem formally. This is a special case of the problem of coloring the vertices of a graph with intervals. Formally, we have an undirected graph $G = (V, E)$ with a weight function on the vertices $w : V \to \mathbb{N}$. The problem is to find a coloring of

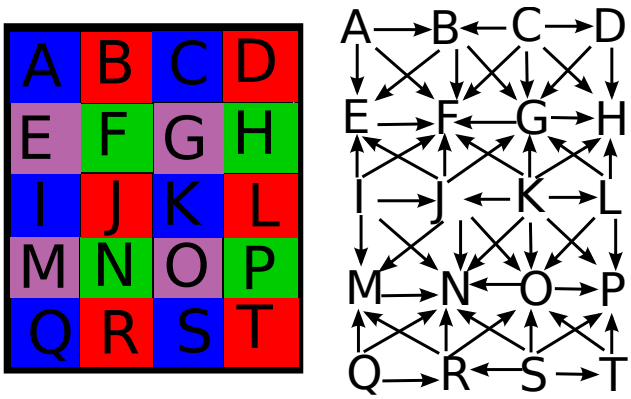


Figure 9: Dependency structure implied by a coloring of the subdomain. (Shown for a 2D decomposition for simplicity.)

each vertex v with $w(v)$ consecutive colors such that for each neighbor $(u, v) \in E$ the colors of u and v are disjoint and such that the number of colors used is minimal. This general problem is NP-Complete since it is a generalization of the classic graph coloring problem [GJ79] and it appears in channel allocation in networking problems. This interval coloring problem is the dual of the edge orientation problem to extract an acyclic graph of minimal length; this result is known as the Gallai-Hasse-Roy-Vitaver theorem for unweighted graph [NOdM12].

While the general problem is NP-Hard, the problem we have is a special case where the graph G is a 3D mesh connected by a 27-point stencil. This could change the complexity of the problem. For instance, the traditional coloring problem (i.e., $\forall v, w(v) = 1$) on such a 3D mesh is polynomial. The complexity of interval coloring on a 3D mesh with 27-point stencil is currently unknown to the authors of this paper. We show two results on this problem. If the graph was a 3D mesh with a 7-point stencil then the problem would be polynomial. Also, the naive coloring described above is a 4-approximation of the optimal solution of the problem.

Theorem 6.1. *If G is a bipartite graph, an optimal coloring can be found in polynomial time.*

Proof. A graph is bipartite if the vertices can be partitioned in two groups A and B such that all edges (u, v) of the graph have $u \in A$ and $v \in B$ (or vice versa).

Consider the following coloring algorithm. All vertices $u \in A$ are colored $[1; w(u)]$. For each vertex of $v \in B$, will be colored $[m(v) + 1; m(v) + w(u)]$ where $m(v)$ is the largest color used by any of the neighbor of v.

If the highest color used comes from vertex $a \in A$, then the number of colors used is $w(a)$ which is a trivial lower bound on the minimum number of colors. If the highest color used comes from vertex $b \in B$, then it uses $w(a) + w(b)$ color which is caused by an edge (a, b) of the graph which is also a lower bound of the minimum number of colors. □

Many structures are bipartite graphs, in particular 1D chains are bipartite, but also 2D 5-point stencils, and 3D 7-point stencils are also bipartite. Unfortunately 2D 9-point stencils and 3D 27-point stencils are not bipartite.

Theorem 6.2. *For a d-dimensional graph connected with a 3^d-point stencil, there exist an approximation algorithm of ratio 2^{d-1}.*

Proof. An algorithm is a ρ approximation algorithm if it is guaranteed to use a number of colors lesser than ρ times the number of colors used by the optimal solution [Hoc97].

The one dimensional case is bipartite, so obviously the algorithm described in the previous theorem applies. For higher dimension the proof is constructed by seeing a d-dimensional mesh as a chain of $d-1$ dimensional meshes and applying the bipartite algorithm.

Let us present the rational in the 2D case. In the 2D case, for each row of the instance, solve the 1D problem using the bipartite algorithm. Each row r uses a number of colors $c(r)$. See the 2D problem has a 1D problem where each vertex is a row r of weight $w(r) = c(r)$ and solve it with the bipartite algorithm. The number of colors used by this algorithm is $w(r) + w(r + 1)$ for a particular row r. The coloring of each row being optimal, both $w(r)$ and

$w(r+1)$ are lower bounds of the original coloring problem; which proves that the derived coloring uses a number of color lesser than 2 times the optimal number of colors.

A proof by induction on higher number of dimensions is easy derived from this structure: a solution at dimension d is constructed from a chain where the weight of each vertex is a solution of the problem of dimension $d-1$ for which a 2^{d-2} approximated solution is known. □

Note that we proved the existence of a 2^{d-1} algorithm but the problem may still be polynomial.

The naive coloring algorithm presented in the previous section that performs an unweighted coloring generate the same solution as the algorithm described in the proof once one shifts the coloring intervals of each vertex to be as soon as possible.

In practice, we can use a greedy coloring algorithm which considers the subdomains in a particular order and for each subdomain will color it with the smallest colors that does not conflict with the already colored neighbors. Such greedy coloring algorithm are commonly used in parallel computing applications [GMP05, BBÇ$^+$05, DBDR16]. However, we will use an ordering that color the vertices in non-increasing order of the number of points contained in the subdomain. While this algorithm is a heuristic, it is likely to obtain a coloring that minimizes the critical path. We call this algorithm to compute STKDE: PB-SYM-PD-SCHED.

Remark that while we expect this greedy algorithm to perform well, it is not optimal even on a 1D chain. Consider the graph composed of four vertices of weight $w(1) = 10$, $w(2) = 9$, $w(3) = 8$, and $w(4) = 10$. The greedy algorithm colors first vertices 1 and 4 with colors $[1; 10]$; then it colors vertex 2 with colors $[11; 19]$; finally it colors vertex 3 with colors $[20; 27]$. Meanwhile, the optimal coloring only uses 18 colors.

6.3 Hybrid algorithms

To decrease the critical path further, we propose an other algorithm PB-SYM-PD-REP which replicates the subdomains that are on the critical path (and its neighbors) of the task graph so that the points of a subdomain can be performed in parallel. As long as the critical path is longer than $\frac{n}{2P}$, the tasks on the path are replicated an additional time and the critical path is recomputed. Replication of subdomain causes to have to initialize additional subdomains and to reduce them which increases the work in a similar way PB-SYM-DR. However, the replication should be limited to a few chains in the graph and therefore should limit the overhead. Note that this problem is similar to scheduling a DAG of moldable tasks [LTW02, Hun13].

7 Experiments

7.1 Experimental Setting

Execution environment The machine used to perform the experiments is composed of two Intel Xeon E5-2667 v3 processors for a total of 16 cores clocked at 3.2Ghz. Despite each core is capable of hosting 2 hyperthreads, hyperthreading was disabled. The machine is equipped with 128GB of DDR4 clocked at 2.133 Ghz. The node uses RHEL 7.2 and Linux 3.10. All codes are written in C++ and are compiled using GCC 5.3, which implements OpenMP 4.0, with optimization flags `-O3 -march=native -fopenmp -DNDEBUG`. All reported execution times exclude all IOs.

Dataset Four datasets are used in our experiments. Their key attributes are summarized in Table 2. Out first dataset, Dengue, is of epidemiological nature and consists of the space-time locations of dengue fever cases reported to the Sistema de Vigilancia en Salud Pública for the city of Cali, Colombia during 2010 and 2011. The system is updated on a daily basis with records of individuals that have been diagnosed with dengue fever, including patient address and the date of diagnosis. Addresses are standardized to a common address format, and spelling and other syntactical errors are manually corrected [DCRV13]; they are then geocoded and masked to the nearest street intersection level to maintain privacy [KCS04]. In 2010, 9606 were successfully geocoded (11,760 reported cases, or 81.7%), whereas in 2011, 1562 cases were successfully geocoded (1822 reported cases, or 85.7%).

The PollenUS dataset is composed of 588K tweets of US users from February 2016 to April 2016 related to Pollen (i.e., that mention keywords such as "pollen" and "allergy"). Tweets that did not contain a precise localization were approximated by picking a random location in the approximated region provided by Gnip[1]. Tweets are mostly located in the contiguous continental US. Therefore, this is the region we modeled with a spatial resolution ranging from $.2^\circ$ to 1° and a temporal resolution of 1 day.

The Flu dataset was obtained from the Animal Surveillance database of the Influenza Research Database[2]. The dataset contains observations of birds tested positive for carrying any strain the avian flu from 2001 to 2016. The birds were observed all over the world and the modeled region spans as West as Alaska and East as Japan, as South as the southern shore of Australia and as North as the Northern shore of Russia. The domain is modeled with spatial resolution from $.2^\circ$ to 1° and a temporal resolution of 1 day.

The eBird[3] dataset is a collection of worldwide rare bird spotting obtained from a crowdsourcing effort managed by Cornell's Lab or Ornithology [CLoO16]. Despite the service was launched in 2002, the database contains many older entries of birds that are known to have been observed at the time. For our experiment, we use the last 20 years of bird spotting and model the entire world with a temporal resolution of 3 days, and a spatial resolution of either $.1^\circ$ or $.5^\circ$.

For all datasets, we created multiple instances of the problem coded Lr, Mr, Hr for low, medium, and high resolution and Lb, Hb, VHb for low, high and very high bandwidth. See details in Table 2.

7.2 Algorithm design

Table 3 presents the runtime of the point-based algorithms presented in Section 3. The runtime are given in seconds. As expected for each dataset, the runtime increases with resolution and bandwidth. Exploiting the disk invariant with PB-DISK provides a large reduction in the runtime, even more so when the temporal bandwidth is high. PB-BAR provides a more modest time reduction.

PB-SYM combines the reductions of both PB-BAR and PB-DISK and achieves the best performance in most cases. PB-SYM provides improvements up to a factor of 6.969 on the PollenUS_Hr-Hb case. The cases where PB-SYM provides little improvement are explained either by the instance having a low bandwidth, and therefore there is little redundant computation to take advantange of, or by the fact that the runtime is dominated by initialization cost. The breakdown of the runtime of PB-SYM is given in Figure 10 and shows that PB-SYM is composed of two phases, initializing the memory to store the density estimation of the voxels, and computing the density cylinder surrounding the points. PB-SYM reduces the computation cost but initializes the memory in the same way as PB. Initialization is a predominant runtime cost in the Flu dataset since there were only 31K occurence of confirmed avian flu recorded in the last 15 years but they span most of the earth.

Comparing to the gold implementation would not be fair because it is in a different programming language. Though we implemented the VB algorithm, and also a variant, VB-DEC, that partitions the points in blocks of size equal to the bandwidth to only compute distances between voxels and points that have a chance to have an impact on it. The results are given in Table 3 and show that even VB-DEC is usually at least an order of magnitude larger than PB and typically two orders of magnitude larger than PB-SYM.

7.3 Domain-based parallelism with PB-SYM-DR

The performance of the PB-SYM-DR algorithm is given in Figure 11. Because it replicates the domain space, PB-SYM-DR was not able to complete the calculations for Flu_Hr-Lb and Flu_Hr-Hb when using 8 threads and 16 threads as the memory requirement exceeds the 128GB available on the machine. None of the high resolution eBird instances could have their domain replicated.

All the instances that have a high initialization cost get a speedup lesser than 1 since the threads spend their time allocating extra memory and reducing the content of that extra memory. The only instances to achieve a speedup higher than 8 when using 16 threads are 3 of the PollenUS instances and one low resolution eBird instances. Not surprisingly, they are the instances with the largest fraction of computation. (See Figure 10 for reference.)

[1] `http://www.gnip.com/`
[2] `https://www.fludb.org/`
[3] `http://ebird.org`

Instance	n	G_x x G_y x G_t	Size	H_s	H_t
Dengue_Lr-Lb	11056	148x194x728	79MB	3	1
Dengue_Lr-Hb	11056	148x194x728	79MB	25	1
Dengue_Hr-Lb	11056	294x386x728	315MB	2	1
Dengue_Hr-Hb	11056	294x386x728	315MB	50	1
Dengue_Hr-VHb	11056	294x386x728	315MB	50	14
PollenUS_Lr-Lb	588189	131x61x84	2MB	2	3
PollenUS_Hr-Lb	588189	651x301x84	62MB	10	3
PollenUS_Hr-Mb	588189	651x301x84	62MB	25	7
PollenUS_Hr-Hb	588189	651x301x84	62MB	50	14
PollenUS_VHr-Lb	588189	6501x3001x84	6252MB	100	3
PollenUS_VHr-VLb	588189	6501x3001x84	6252MB	50	3
Flu_Lr-Lb	31478	117x308x851	117MB	1	1
Flu_Lr-Hb	31478	117x308x851	117MB	2	3
Flu_Mr-Lb	31478	233x615x1985	1085MB	2	3
Flu_Mr-Hb	31478	233x615x1985	1085MB	4	7
Flu_Hr-Lb	31478	581x1536x5951	20260MB	5	7
Flu_Hr-Hb	31478	581x1536x5951	20260MB	10	21
eBird_Lr-Lb	291990435	357x721x2435	2391MB	2	3
eBird_Lr-Hb	291990435	357x721x2435	2391MB	6	5
eBird_Hr-Lb	291990435	1781x3601x2435	59570MB	10	3
eBird_Hr-Hb	291990435	1781x3601x2435	59570MB	30	5

Table 2: Properties of the datasets. The domain is expressed in voxels and in MB. Bandwidth are expressed in voxels.

Instance	Time (in seconds) VB	VB-DEC	PB	PB-DISK	PB-BAR	PB-SYM	speedup PB-SYM
Dengue_Lr-Lb	219.163	2.283	0.040	0.029	0.035	0.028	1.429
Dengue_Lr-Hb	220.591	13.878	1.298	0.564	1.152	0.499	2.601
Dengue_Hr-Lb	866.445	9.522	0.089	0.082	0.085	0.084	1.060
Dengue_Hr-Hb	871.774	55.206	5.169	2.272	4.563	2.074	2.492
Dengue_Hr-VHb	1056.172	404.845	51.885	11.478	42.994	7.431	6.982
PollenUS_Lr-Lb	518.859	7.639	1.106	0.347	0.922	0.256	4.320
PollenUS_Hr-Lb	12721.001	189.337	23.539	7.700	18.527	4.708	5.000
PollenUS_Hr-Mb	17179.482	3126.947	357.743	86.129	295.791	57.528	6.219
PollenUS_Hr-Hb			2666.104	583.175	2212.626	382.566	6.969
PollenUS_VHr-Lb			2428.126	1004.174	1949.988	759.722	3.196
PollenUS_VHr-VLb			603.789	240.236	488.388	179.834	3.357
Flu_Lr-Lb	926.360	3.691	0.035	0.032	0.034	0.032	1.094
Flu_Lr-Hb	966.328	3.797	0.081	0.046	0.070	0.042	1.929
Flu_Mr-Lb	8591.165	30.355	0.305	0.278	0.298	0.277	1.101
Flu_Mr-Hb	8957.175	32.018	0.714	0.384	0.608	0.323	2.211
Flu_Hr-Lb		536.091	5.702	5.089	5.454	5.059	1.127
Flu_Hr-Hb		591.955	12.795	6.822	10.992	7.072	1.809
eBird_Lr-Lb			396.811	147.951	322.580	125.248	3.168
eBird_Lr-Hb			6969.187	1897.051	5611.158	1067.395	6.529
eBird_Hr-Lb			8373.273	3226.016	6470.764	2229.460	3.756
eBird_Hr-Hb						34577.745	

Table 3: Runtime of different algorithm. The speedup of PB-SYM over PB is given.

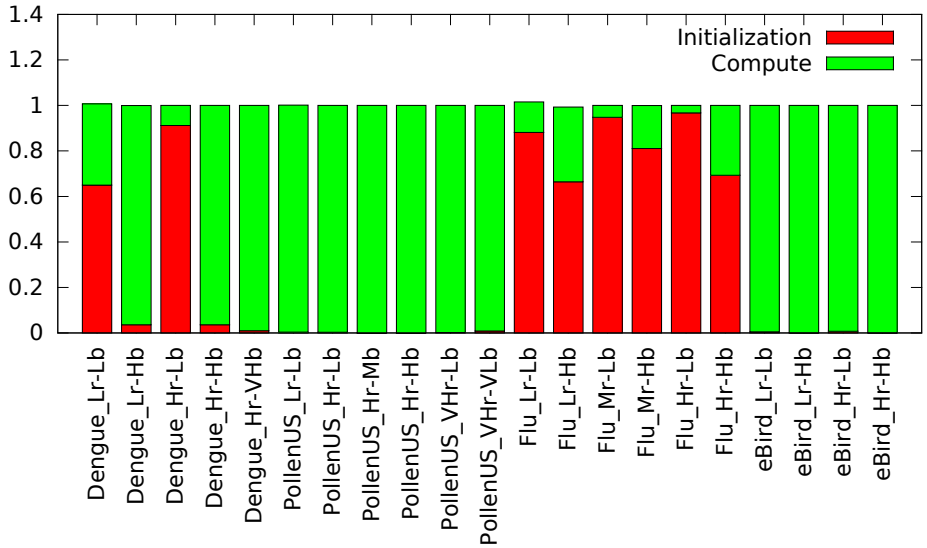


Figure 10: Breakdown of the runtime of PB-SYM. Some instances are mostly Initialization.

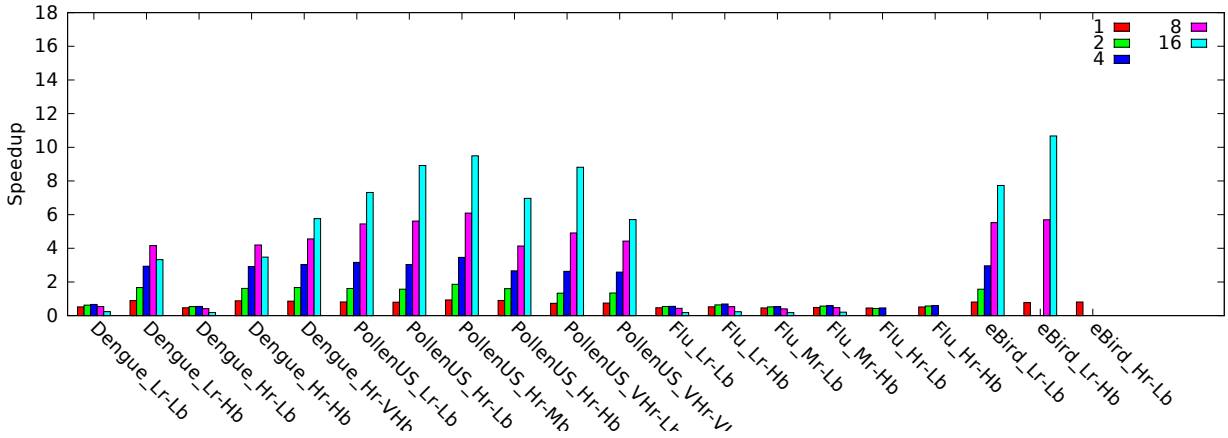


Figure 11: Speedup of PB-SYM-DR for different number of threads used. Some instances run out of memory.

7.4 Domain-based parallelism with PB-SYM-DD

7.4.1 With Regular Grid Partitioning

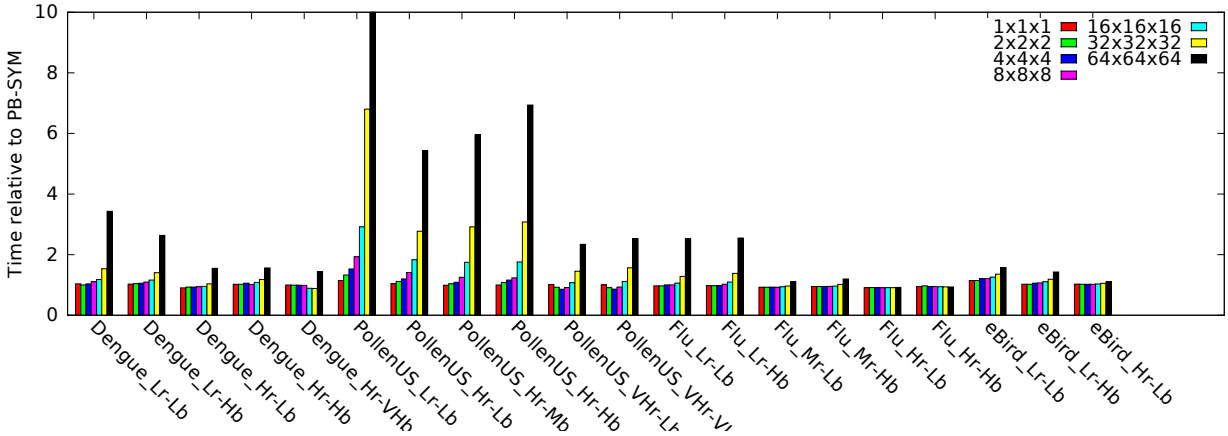


Figure 12: Overhead of PB-SYM-DD: runtime of PB-SYM-DD with a single thread normalized to PB-SYM. Some decomposition improve performance dues to better cache fitting. Overdecomposition can cause significant overhead.

The PB-SYM-DD algorithm (described in Section 5) also has some overhead induced by cut cylinder surrounding a point. To measure the overhead beforehand, we run the decomposition algorithm using different number of subdomains ranging from a single subdomain (1x1x1) to a very fine grain decomposition in 64x64x64 subdomains (except on the eBird_Hr_Hb where such a test is computationally expensive). The results of that experiment is presented in Figure 12. The figure presents the overhead of the decomposition relatively to PB-SYM. Two things are apparent. Some decomposition can actually reduce the runtime by providing better cache locality; for instance Flu_Hr-Lb is 9.8% faster using a 16x16x16 decomposition than PB-SYM.

However, in most cases, the decomposition actually increases the runtime. For instance, a 16x16x16 decomposition increases the amount of work by 9% in the Flu-Lr-Hb cases, and a 64x64x64 decomposition induces an overhead of 254%. Even on the Dengue_Lr-Hb, a simple 4x4x4 decomposition already induces a 5% overhead which grows to 69%

for a 16x16x16 decomposition. Such large decomposition are likely to be necessary since the points in the datasets are often organized in clusters: a decomposition in 16 subdomains is unlikely to provide a load balanced execution. The PollenUS suffers from the highest overheads, with a 16x16x16 decomposition leading to a 74% overhead on PollenUS-Hr-Mb, and a 495% overhead for a 64x64x64 decomposition

The speedup achieved by a parallel execution of PB-SYM-DD using 16 threads is presented in Figure 13. This algorithm achieves overall better speedup than PB-SYM-DR. It achieves a speedup greater than 8 on 9 instances. Let us note that a speedup of 14.9 is achieved on Dengue_Hr-VHb using a 16x16x16 decomposition and of 14.8 on eBird_Hr-Hb with a 32x32x32 decomposition.

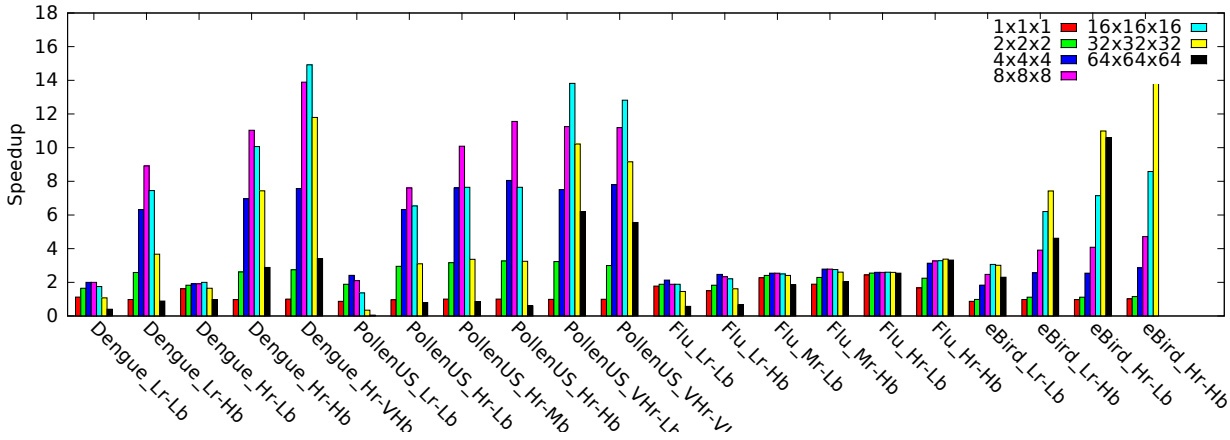


Figure 13: Speedup of PB-SYM-DD with 16 threads. While PB-SYM-DD achieves good speedup in some cases, the overdecomposition overhead usually hinders the performance.

Despite a significant overhead on the PollenUS_Hr instances, a speedup of 10 and 11.5 can be observed on the Mb and Hb cases using 8x8x8 decomposition. This speedup is relatively impressive provided the decomposition induces an overhead of 25% and 23% in these two cases. The 8x8x8 decomposition in that case is still imbalanced and a finer decomposition can reach perfect load balance; however the overhead induced by the decomposition prevents taking advantages of that better load balance. For PB-SYM-DD to be useful, it is important to pick the decomposition to ensure load balance without suffering from an unbearable computational overhead.

A more modest speedup between 2 and 4 can be observed on the instances with higher initialization cost, such as on the Flu dataset. Indeed, most of the time is spent on initializing the memory to hold the density estimates. Even if the algorithm performs memory initialization in parallel, there is not much room for parallelism in memory operations. Even more so since the initialization touches the memory for the first time, which makes the operating system allocates the pages in physical memory. The speedup of the initialization phase using 16 threads is about the same for all instances and is at about 3. The small speedup of PB-SYM-DD on instances with high initialization cost is completely explained by this phenomenon. Even if the compute phase was reduced to 0, the speedup on the entire calculation would only be 3.7.

7.4.2 Considering better partitioning.

We computed partitionings for 16, 32, and 64 sub-domains. And using $Xstep$, $Ystep$, and $Tstep$ values of 4, 8, 16, 32, 64, 256, and 512. For over-decomposition methods, we set the constraint by using the ratio between the largest allowed box and a perfect partitioning. We used ratios of 1.1, 1.5, 2, 3 and 4. Unfeasible partitioning were discarded. We also discarded all partitioning which took more than 2 hours to compute.

We report the quality of the partitioning in Figure 14 for Jagged partitioning and in Figure 15. On each figure, we are also showing a simple grid based over-decomposition that decomposed the space along each axis in 4, 8, 16, 32, and 64 blocks. On the x-axis, each plot shows normalized values for $1/p_{max}$. In other words, if $1/p_{max} = 16$,

one would expect to see a speedup of 16 if there was only the largest sub-domain to process. On the y-axis, the plots show the normalized work overhead, a decomposition that introduces no additional work would have a work overhead of 0 (there are none of those reported) while a decomposition that double the amount of work would have an work overhead of 1. As such, one wants to achieve a partitioning with a high $1/p_{max}$ value and a low work overhead, and therefore wishes to be the most on the bottom right of the charts.

The first observation one can have is that a grid based partitioning consistently generates highly work-inefficient partitiong with over overheads often larger than 1. As a comparison most of the other methods tend to return partitioning with work overhead less than .25. The solutions computed by Jagged partitioning or Hierarchical decomposition that have a high work overhead and a low $1/p_{max}$ are derived from high $Xstep$, $Ystep$, and $Tstep$ values and therefore have lead to very inefficient partitioning.

For Jagged partitioning, the over decomposition methods most of the time exposes more interesting trade-off than the partitioning generated by the exact decomposition methods; typically on Dengue-Hr-Hb. Few cases of the exact partitioning leading to more interesting trade-off than the over decomposition method can be seen, for instance on PollenUS-VHr-Lb. Upon analysis, these cases come from the exact method converging faster than the over decomposition method and returning a solution for smaller values of $Xstep$, $Ystep$, and $Tstep$, leading to a finer level of optimization. (The similar cases for the over decomposition timed out).

The Hierarchical decomposition show a slightly different picture. In particular, more cases of the over decomposition timed out. This leads to fewer interesting cases of over-decomposition. Also, interestingly, the over decomposition returned the same solution as the exact decomposition in many cases; which did not happen for the Jagged decomposition. This makes some intuitive sense as hierarchical partitioning have more leeway to optimize the structure of the solution and is less likely to be forced to pick a decomposition that increase the amount of work significantly. In any cases, the work aware method presented in this manuscript lead to much better decomposition than the naive grid partitioning.

7.4.3 Real execution

Some of the partitioning generated by the different methods were plugged in a parallel code of STKDE to see how much speedup these improved partition would obtain. The results at this point are preliminary and therefore the details are excluded from the manuscript as only few of the configuration have actually been run. However, the author can confirm that the gains in the quality of the partitioning do in practice accelerate for the STKDE application.

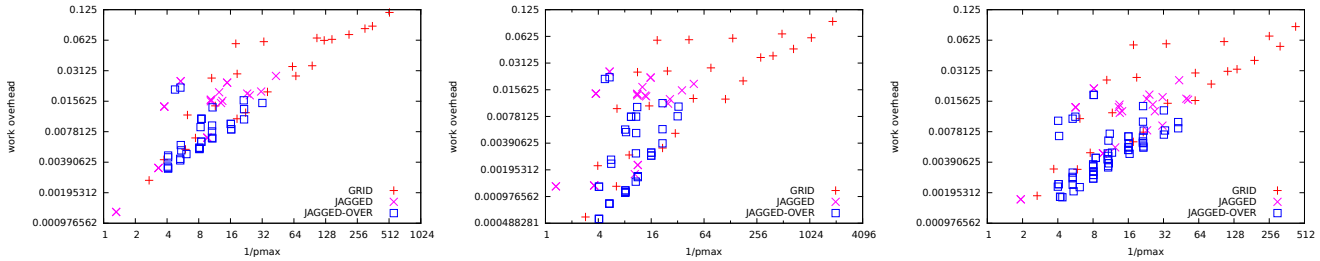
7.5 Point-based parallelism

The performance of PB-SYM-PD is shown in Figure 18 for different decomposition. Note that there is a requirement in PB-SYM-PD that the subdomain be larger than twice the bandwidth (see details in Section 6). Therefore if the number of subdomain leads to too small subdomains, the decomposition is adjusted to fit the required size.

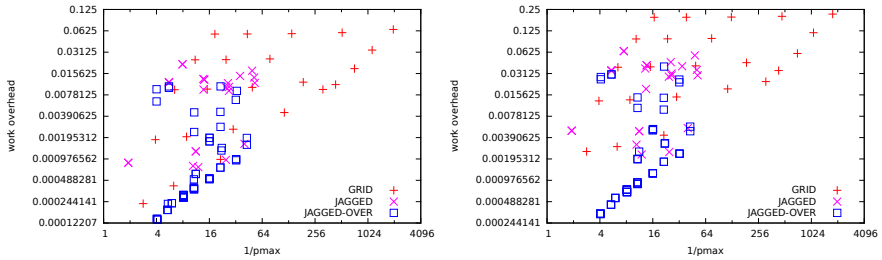
PB-SYM-PD does not scale well on all instances. The highest speedup achieved on PollenUS_Lr-Lb is 2.6. It is interesting to notice that the speedup usually increases with decomposition size. However, the speedup is limited because of load imbalance. Figure 19 shows the implicit critical path within the PB-SYM-PD algorithm for the highest decomposition used in the experiment (64x64x64). Most instances have a critical path of about 10% of the total work. If Graham's bound is reached, that implies a speedup lesser than 6.15. PollenUS_Hr-Hb has a critical path of 55% which implies a speedup lesser than 1.6. However, the speedup is typically lower because of the limited scalability in the initialization phase.

PB-SYM-PD-SCHED aims at improving the load balance by arranging the execution to process the most loaded subdomain first. Figure 19 shows that PB-SYM-PD-SCHED marginally decreases the critical path in all but one case. Even if that decrease is marginal, PB-SYM-PD-SCHED ensures that the highest loaded subdomain are executed first which tends to construct a better execution in practice. And one can see from Figure 20 that this scheduling improves parallelism significantly for the PollenUS instance. A superlinear speedup occurs on PollenUS_VHr-VLb: it is due to the decomposition of the points which improves the locality of the computation compared to the sequential execution.

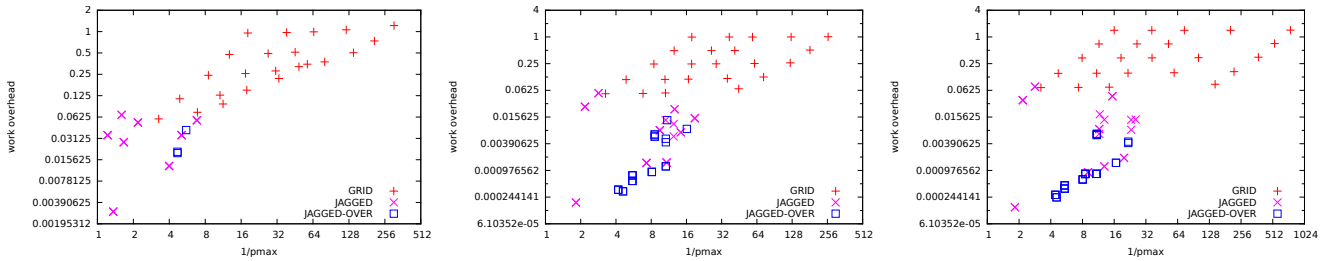
However, the parallelism is still constrained by load imbalance. PB-SYM-PD-REP performs domain decomposition on the subdomain of the most loaded chains in the dependency graph until the critical is small. Figure 21 shows the speedup achieved by PB-SYM-PD-REP. Using PB-SYM-PD-REP achieves a speedup larger than 8 on 8 instances.



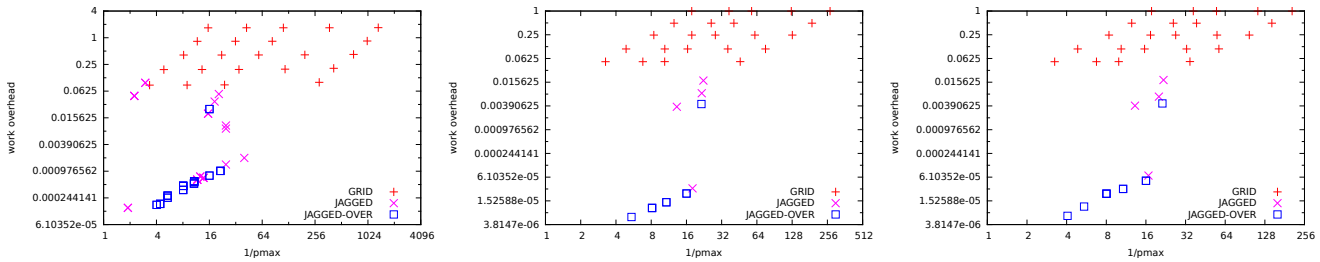
(a) Dengue-Lr-Lb (b) Dengue-Lr-Hb (c) Dengue-Lr-Hb



(d) Dengue-Hr-Hb (e) Dengue-Hr-VHb

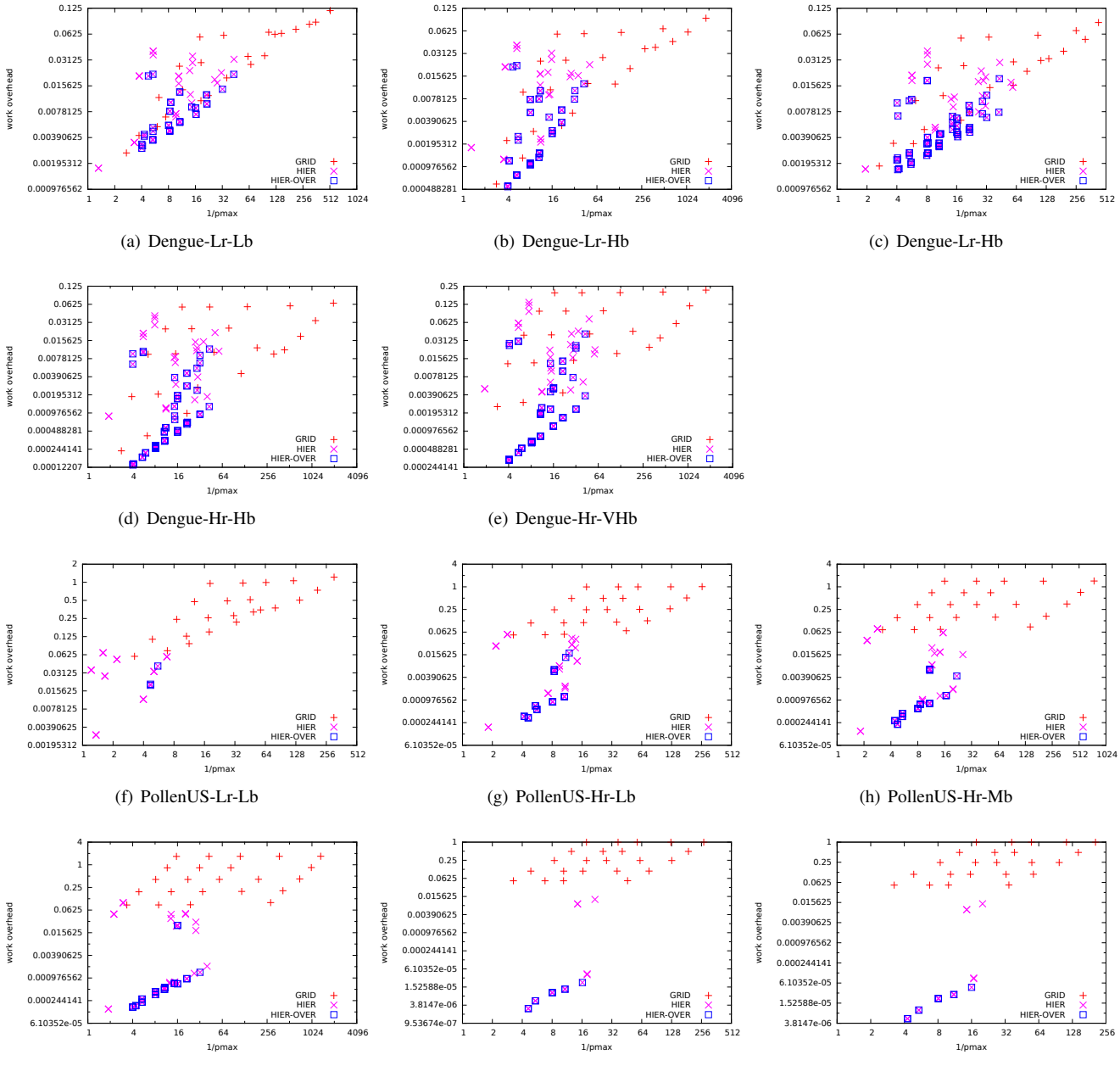


(f) PollenUS-Lr-Lb (g) PollenUS-Hr-Lb (h) PollenUS-Hr-Mb



(i) PollenUS-Hr-Hb (j) PollenUS-VHr-Lb (k) PollenUS-VHr-VLb

Figure 14: Quality of partitioning generated by Jagged decomposition



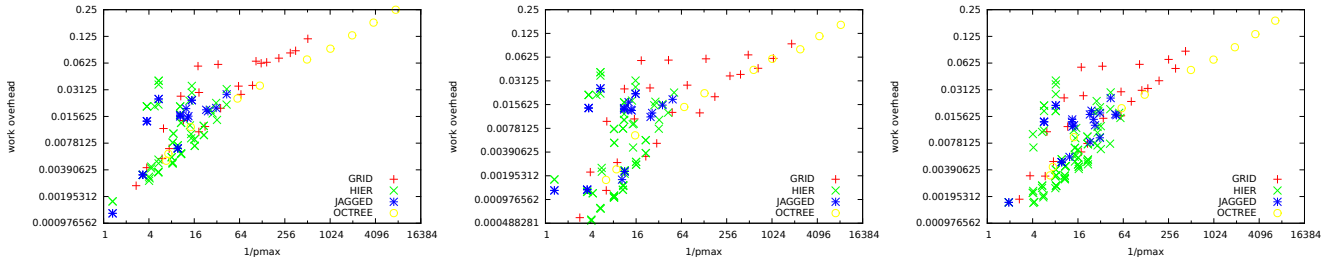
(a) Dengue-Lr-Lb (b) Dengue-Lr-Hb (c) Dengue-Lr-Hb

(d) Dengue-Hr-Hb (e) Dengue-Hr-VHb

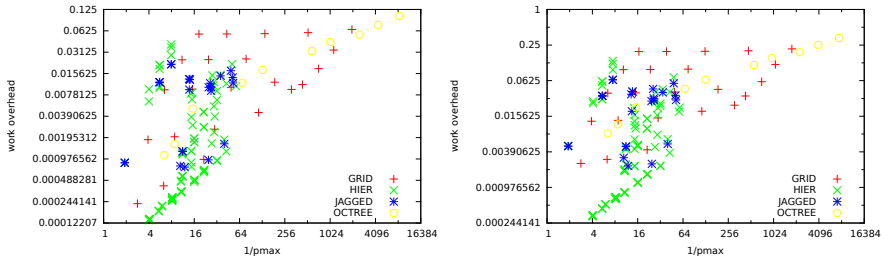
(f) PollenUS-Lr-Lb (g) PollenUS-Hr-Lb (h) PollenUS-Hr-Mb

(i) PollenUS-Hr-Hb (j) PollenUS-VHr-Lb (k) PollenUS-VHr-VLb

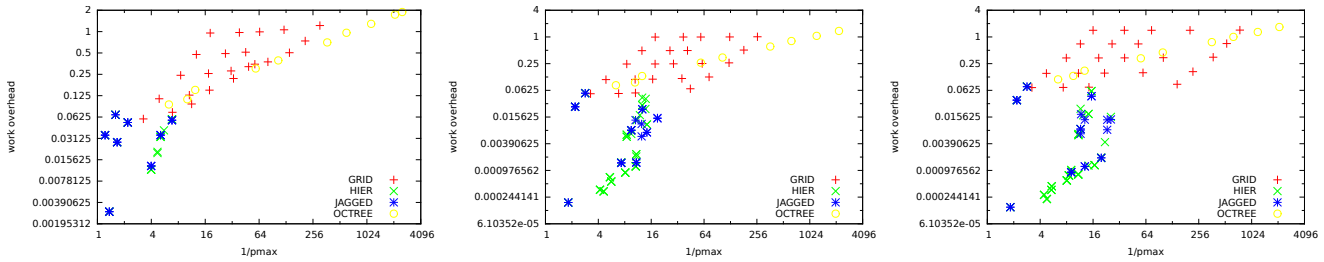
Figure 15: Quality of Partitionings generated by Hierarchical Decomposition



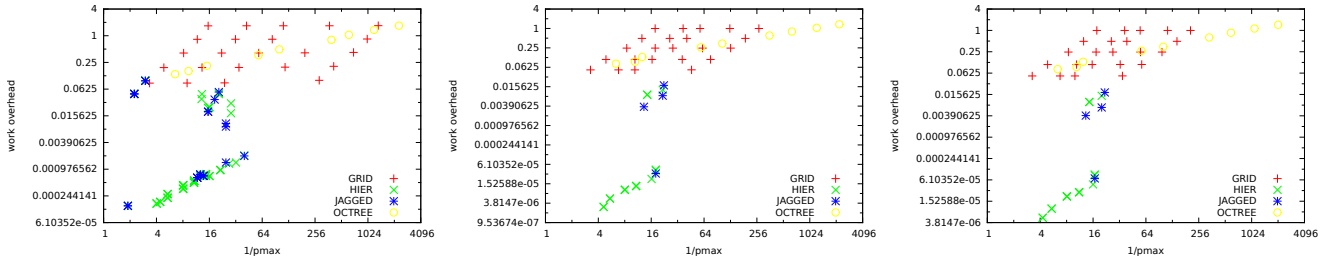
(a) Dengue-Lr-Lb (b) Dengue-Lr-Hb (c) Dengue-Lr-Hb



(d) Dengue-Hr-Hb (e) Dengue-Hr-VHb

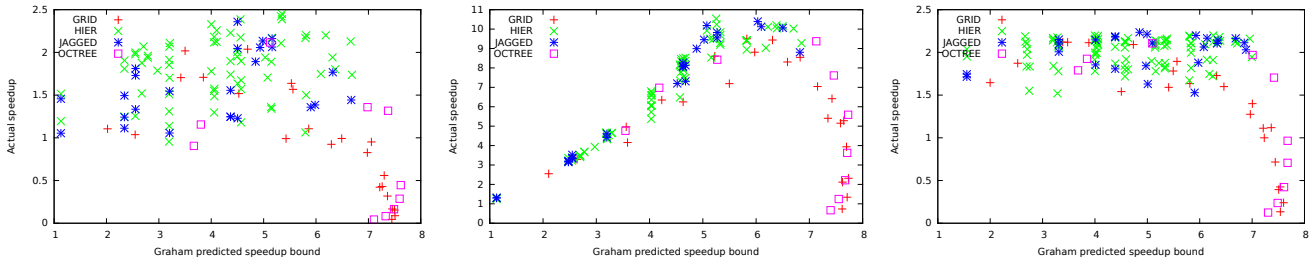


(f) PollenUS-Lr-Lb (g) PollenUS-Hr-Lb (h) PollenUS-Hr-Mb

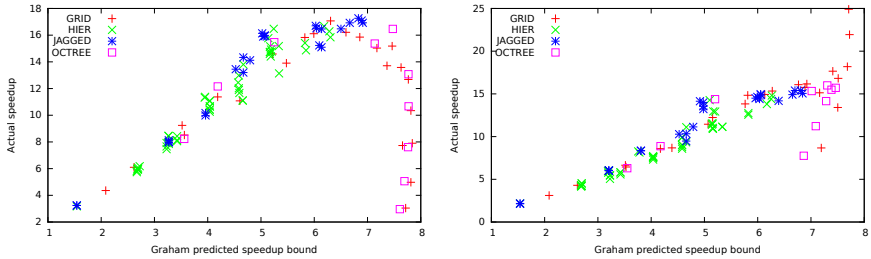


(i) PollenUS-Hr-Hb (j) PollenUS-VHr-Lb (k) PollenUS-VHr-VLb

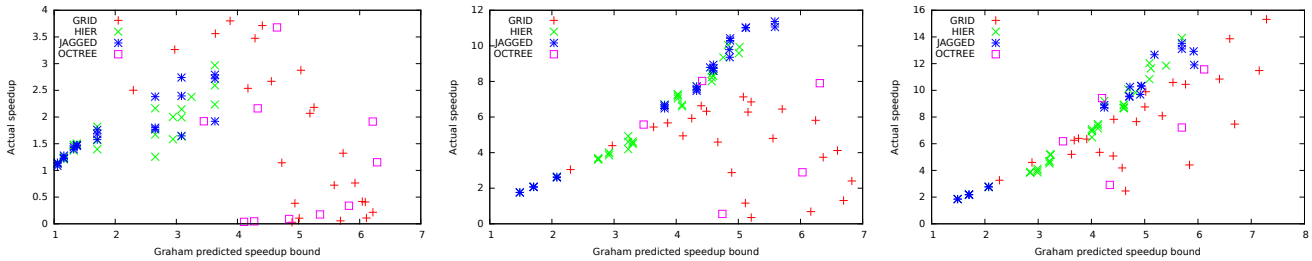
Figure 16: Quality of all Partitionings



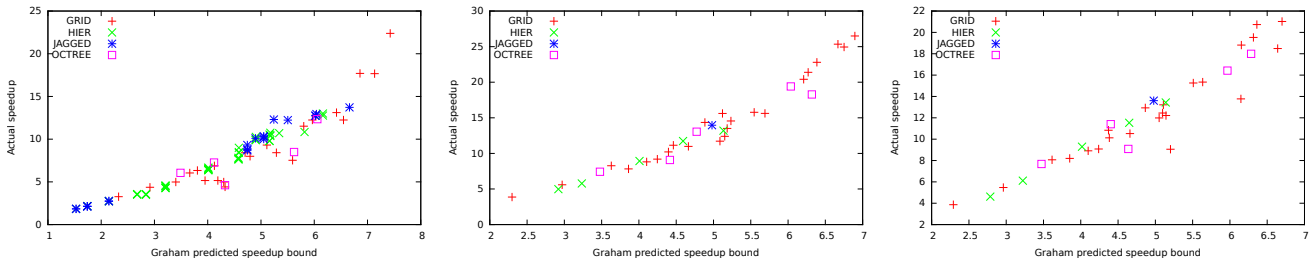
(a) Dengue-Lr-Lb (b) Dengue-Lr-Hb (c) Dengue-Lr-Hb



(d) Dengue-Hr-Hb (e) Dengue-Hr-VHb



(f) PollenUS-Lr-Lb (g) PollenUS-Hr-Lb (h) PollenUS-Hr-Mb



(i) PollenUS-Hr-Hb (j) PollenUS-VHr-Lb (k) PollenUS-VHr-VLb

Figure 17: actual run of all Partitionings

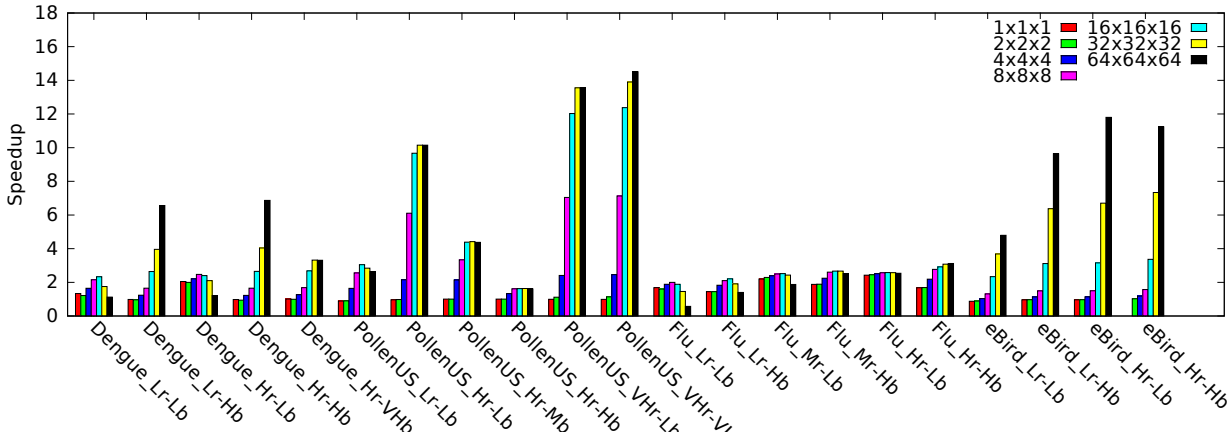


Figure 18: Speedup of PB-SYM-PD using 16 threads. Note that decompositions of subdomain smaller that twice the bandwidths are adjusted.

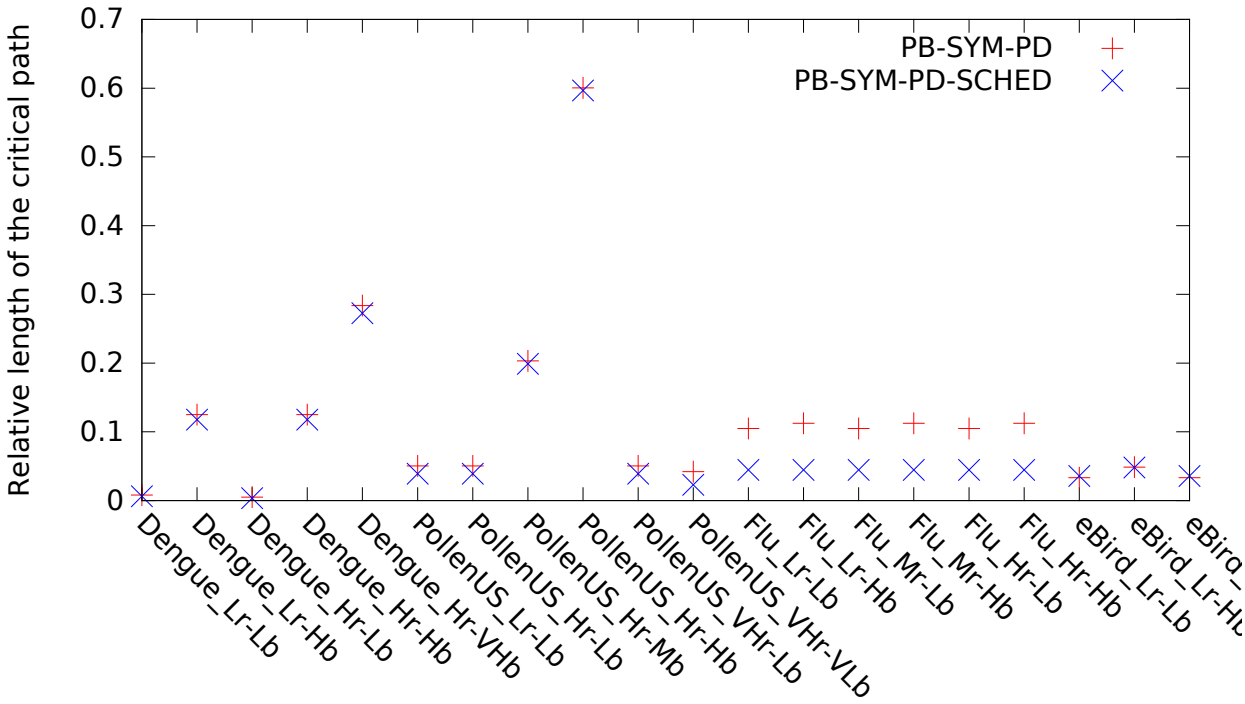


Figure 19: Length of the critical path of the parallel algorithms PB-SYM-PD and PB-SYM-PD-SCHED for a 64x64x64 decomposition

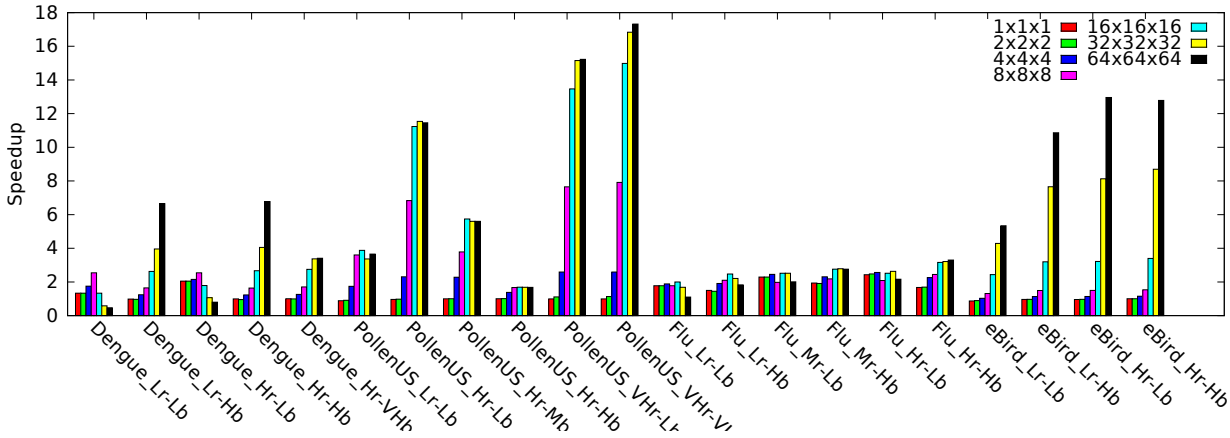


Figure 20: Speedup of PB-SYM-PD-SCHED with 16 threads.

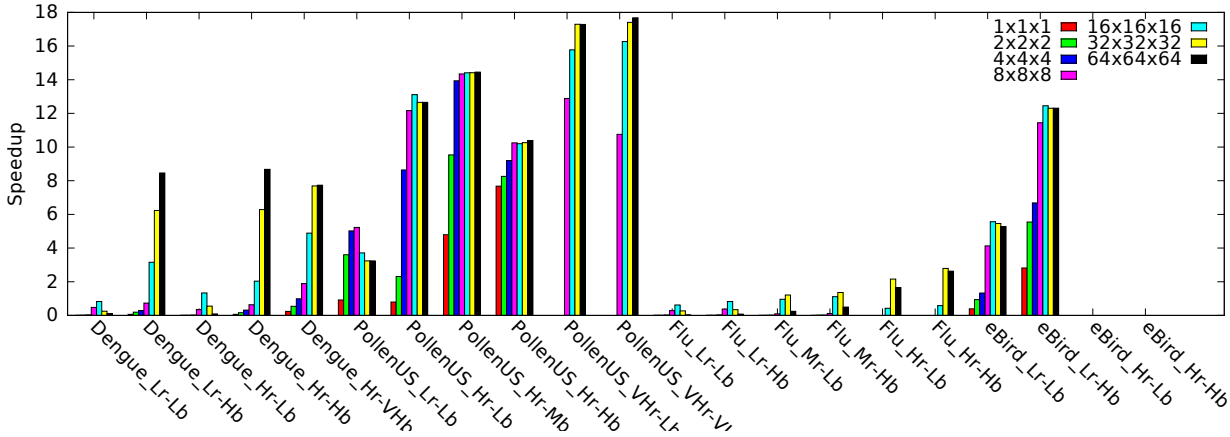


Figure 21: Speedup of PB-SYM-PD-REP using 16 threads for different decompositions. Flu_Hr-Lb and Flu_Hr-Hb run out of memory for small decomposition.

Though the speedup is close to 0 for small decompositions and on certain instances. Indeed, when there is no (or little) decomposition, most of the domain is replicated and the execution has similar drawbacks as PB-SYM-DR.

7.6 Summary and Discussion

Figure 22 summarizes the performance of all our methods. It shows the performance achieved by the best configuration of each particular algorithm. On Dengue instances, PB-SYM-DD usually leads to the best performance because of its low overhead on that instance while maintaining a good load balance. On the PollenUS instances, the smart scheduling of PB-SYM-PD-SCHED-REP is necessary to reach the highest parallelism. The flu instances are mostly dominated to initialization overhead because it is very sparse, as such, PB-SYM-DR performs much worse than the other methods, and past that issue, the method leads to little performance difference. The eBird instances have a very small memory initialization overhead because they are dense in computation. This makes approaches that replicate the voxel space perform well at low resolution, but runs out of memory at high resolution there.

What we need to do is to develop a parametric model for the problem that will take into account memory avail-

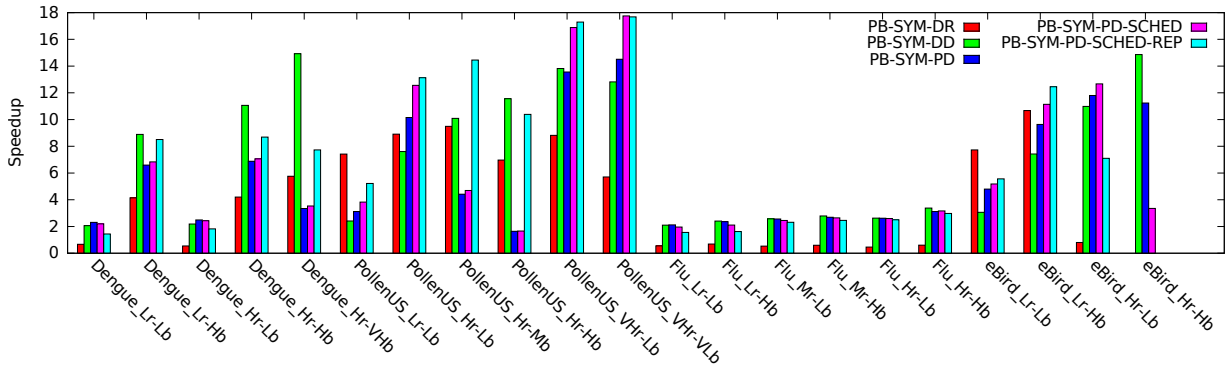


Figure 22: Best configurations

ability, cost of memory initialization, expected cost of computing the kernel density. Using that model finding the best execution strategy becomes a combinatorial problem.

8 Related Works

The PhD Thesis of U. Lopez-Novoa [Lop15] is on the topic of computing kernel density estimation for arbitrarily shaped kernels. The crop and chop method presented revolves around cropping the voxel space around a point with similar goal as PB, and compute the density estimates in parallel on a multi-core CPU or on a GPU before the data is aggregated back on the CPU. The arbitrarily shaped density considered does not expose the invariants leveraged by PB-SYM which makes their work inapplicable to STKDE. [ECR17] considers the same variant of kernel density that we consider and expand the techniques of [Lop15] by considering coalescing. However, they still use a voxel based algorithm which is very slow (on a resolution lower than our lowest resolution, the parallel computation takes .6 seconds on a GPU which is slower than our reported sequential time on a larger instance).

The two most common spatial problems solved with parallel computing are N-body interactions [Aar03] and Particle in Cell simulations [KVKVO06]. STKDE is different in the sense that there is no iterating over the problem multiple times as we see in the timesteps of N-body or particle in cell, no direct interaction between points, and no field updates as in Particle in Cell (unless you consider a field discretized at a one voxel size which makes the analogy not quite useful).

The summation of kernel function, in particular of radial basis function attracted some attention [MXYB15, FLF11, YDGD03]. While some method presented in these papers are similar to some of the decomposition we discuss here, the mathematical property of the kernel function are different. In particular the kernel functions that these papers consider do not have the grid-aligned symmetries that the space-time kernel density estimate have. That symmetry is what we leverage in PB-SYM to gain an order of magnitude of acceleration and that leads to a more complex management of parallelism we investigated.

Spatial applications can use spatial partitioning techniques such as recursive bisection [BB87], jagged partition [USZS96, PA97, DRDÇ16], or rectilinear partition [Nic94, MS96]. Depending on the algorithms used for STKDE, the objective function is different. A partitioning for PB-SYM-DD needs a good load balance, to minimize the number of cut cylinders. But PB-SYM-PD needs a decomposition where the subdomains have a minimum size and the load balance critirion is harder to express since neighboring subdomains can not be processed simultaneously.

9 Conclusion

We presented in this paper the space-time kernel density estimation problem which is useful in the visualization of events located in space and time. We proposed sequential algorithms to decrease the complexity of the problem. And we investigated four parallel algorithms, two of which are pleasingly parallel but at the cost of not being work efficient.

We then designed a work-efficient parallel algorithm but the dependency structure tends to prevent high degree of parallelism. Using graph coloring and moldable scheduling techniques we made that latter parallel algorithm more parallel by adding some work-overhead.

For each instance of the problem, one of the parallel algorithm achieved an interesting performance. However, it is clear that we need to model the instance and the platform to control the various overhead and be able to pick the parallel strategy that will derive the highest performance. It would also be interesting to look at distributed memory machines and accelerators to reduce the runtime further since real-time is desirable for interactive applications. In term of the application, we would like to investigate how these methods apply to a bandwidth that adapts to the density of population of the area is also of interest.

Acknowledgment

The authors would like to thank Dr. Daniel Janies for pointing us to the Flu dataset and to thank Bora Uçar and Julien Hermann for some discussion on the partitioning and coloring problem. Support from US NSF XSEDE Supercomputing Resource Allocation (SES170007) "Accelerating and enhancing multi-scale spatiotemporally explicit analysis and modeling of geospatial systems" is acknowledged. This material is based upon work supported by the National Science Foundation under Grant No. 1652442.

References

[Aar03] Sverre J. Aarseth. *Gravitational N-Body Simulations: Tools and Algorithms*. Cambridge University Press, 2003.

[AHM98] Bengt Aspvall, Magnús M. Halldórsson, and Fredrik Manne. Approximations for the general block distribution of a matrix. In *SWAT '98: Proceedings of the 6th Scandinavian Workshop on Algorithm Theory*, pages 47–58, London, UK, 1998. Springer-Verlag.

[BB87] Marsha Berger and Shahid Bokhari. A partitioning strategy for nonuniform problems on multiprocessors. *IEEE TC*, C36(5):570–580, 1987.

[BBÇ+05] E.G. Boman, D. Bozdağ, Ü.V. Çatalyürek, A.H. Gebremedhin, and F. Manne. A scalable parallel graph coloring algorithm for distributed memory computers. In *Proc. of Euro-Par*, pages 241–251, Aug 2005.

[Boa13] OpenMP Architecture Review Board. Openmp application program interface. Technical report, OpenMP Architecture Review Board, July 2013. Verion 4.0.0.

[CLoO16] New York Cornell Lab of Ornithology, Ithaca. ebird basic dataset. http://ebird.org/content/ebird/, May 2016. Version: EBD_relMay-2016.

[DBDR16] M. Deveci, E. G. Boman, K. D. Devine, and S. Rajamanickam. Parallel graph coloring for manycore architectures. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 892–901, May 2016.

[DCRV13] Eric Delmelle, Irene Casas, Jorge H Rojas, and Alejandro Varela. Spatio-temporal patterns of dengue fever in cali, colombia. *International Journal of Applied Geospatial Research (IJAGR)*, 4(4):58–75, 2013.

[DDC+14] E. Delmelle, C. Dony, I. Casas, M. Jia, and W Tang. Visualizing the impact of space-time uncertainties on dengue fever patterns. *International Journal of Geographical Information Science*, 28(5):1107–1127, 2014.

[DRDÇ16] Mehmet Deveci, Sivasankaran Rajamanickam, Karen D. Devine, and Ümit V. Çatalyürek. Multi-jagged: A scalable parallel spatial partitioning algorithm. *IEEE Trans. Parallel Distrib. Syst.*, 27(3):803–817, 2016.

[ECR17] Todd Eaglin, Isaac Cho, and William Ribarsky. Space-time kernel density estimation for real-time interactive visual analytics. In *Proceedings of the 50th Hawaii International Conference on System Sciences*, January 2017.

[EE11] Lars Eisen and Rebecca J Eisen. Using geographic information systems and decision support systems for the prediction, prevention, and control of vector-borne diseases. *Annual review of entomology*, 56:41–61, 2011.

[FLF11] Bengt Fornberg, Elisabeth Larsson, and Natasha Flyer. Stable computations with gaussian radial basis functions. *SIAM Journal on Scientific Computing*, 33(2):869–892, 2011.

[GJ79] Michael R. Garey and David S. Johnson. *Computers and Intractability*. Freeman, San Francisco, 1979.

[GM96] Michelangelo Grigni and Fredrik Manne. On the complexity of the generalized block distribution. In *Proc. of IRREGULAR '96*, pages 319–326, 1996.

[GMP05] Assefaw H. Gebremedhin, Fredrik Manne, and Alex Pothen. What color is your jacobian? Graph coloring for computing derivatives. *SIAM Review*, 47(4):629–705, 2005.

[Gra66] Ronald L. Graham. Bounds for certain multiprocessing anomalies. *Bell System Technical Journal*, 45:1563–1581, 1966.

[Gra69] Ronald L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, March 1969.

[GWM14] Tony H Grubesic, Ran Wei, and Alan T Murray. Spatial clustering overview and comparison: Accuracy, sensitivity, and computational expense. *Annals of the Association of American Geographers*, 104(6):1134–1156, 2014.

[HDTC16] A. Hohl, E. Delmelle, W. Tang, and I. Casas. Accelerating the discovery of space-time patterns of infectious diseases using parallel computing. *Spatial and Spatio-temporal Epidemiology*, 2016.

[Hoc97] Dorit S. Hochbaum, editor. *Approximation Algorithms for NP-hard problems*. PWS publishing company, 97.

[Hun13] Sascha Hunold. Scheduling moldable tasks with precedence constraints and arbitrary speedup functions on multiprocessors. In *Prof of PPAM*, pages 13–25, 2013.

[KCS04] M. P. Kwan, I. Casas, and B. Schmitz. Protection of geoprivacy and accuracy of spatial information: how effective are geographical masks? *Cartographica: The International Journal for Geographic Information and Geovisualization*, 39(2):15–28, 2004.

[KVKVO06] H. Karimabadi, H. X. Vu, D. Krauss-Varban, and Y. Omelchenko. Global hybrid simulations of the earth's magnetosphere. *Numerical Modeling of Space Plasma Flows*, December 2006.

[Leu04] J. Y-T. Leung, editor. *Handbook of Scheduling*. CRC Press, 2004.

[Lop15] Unai Lopez-Novoa. *Contributions to the Efficient Use of General Purpose Coprocessors: Kernel Density Estimation as Case Study*. PhD thesis, Universidad del País Vasco, 2015.

[LTW02] Renaud Lepère, Denis Trystram, and Gerhard J. Woeginger. Approximation algorithms for scheduling malleable tasks under precedence constraints. *Int. J. Found. Comput. Sci.*, 13(4):613–627, 2002.

[MS96] Fredrik Manne and Tor Sørevik. Partitioning an array onto a mesh of processors. In *Proc of PARA '96*, pages 467–477, 1996.

[MXYB15] William B March, Bo Xiao, Chenhan D Yu, and George Biros. ASKIT: an efficient, parallel library for high-dimensional kernel summations. *SIAM Journal on Scientific Computing (to appear)*, 2015.

[Nic94] David Nicol. Rectilinear partitioning of irregular data parallel computations. *JPDC*, 23:119–134, 1994.

[NOdM12] Jaroslav Nešetřil and Patrice Ossona de Mendez. *Sparsity: – Graphs, Structures, and Algorithms*, volume 28 of *Algorithms and Combinatorics*. Springer-Verlag Berlin Heidelberg, 2012.

[NY10] T. Nakaya and K. Yano. Visualising crime clusters in a space-time cube: an exploratory data-analysis approach using space-time kernel density estimation and scan statistics. *Transactions in GIS*, 14(3):223–239, 2010.

[PA97] Ali Pınar and Cevdet Aykanat. Sparse matrix decomposition with optimal load balancing. In *Proc. of HiPC 1997*, 1997.

[PA04] Ali Pınar and Cevdet Aykanat. Fast optimal load balancing algorithms for 1D partitioning. *Journal of Parallel and Distributed Computing*, 64:974–996, 2004.

[PTA08] A. Pınar, E.K. Tabak, and C. Aykanat. One-dimensional partitioning for heterogeneous systems: Theory and practice. *Journal of Parallel and Distributed Computing*, 68:1473–1486, 2008.

[PY00] Christos H. Papadimitriou and Mihalis Yannakakis. On the approximability of trade-offs and optimal access of web sources. In FOCS, editor, *41st Annual Symposium on Foundations of Computer Science*, pages 86–92, 2000.

[SBC12] Erik Saule, Erdeniz O. Bas, and Umit V. Catalyurek. Load-balancing spatially located computations using rectangular partitions. *Journal of Parallel and Distributed Computing*, 2012.

[Sil86] B. W. Silverman. *Density estimation for statistics and data analysis*, volume 26. CRC press, 1986.

[SPH+17] Erik Saule, Dinesh Panchananam, Alexander Hohl, Wenwu Tang, and Eric Delmelle. Parallel space-time kernel density estimation. In *Proc of ICPP 2017*, 2017.

[USZS96] Manuel Ujaldon, Shamik Sharma, Emilio Zapata, and Joel Saltz. Experimental evaluation of efficient sparse matrix distributions. In *Proc. of SuperComputing'96*, 1996.

[YDGD03] Changjiang Yang, Ramani Duraiswami, Nail A Gumerov, and Larry Davis. Improved fast gauss transform and efficient kernel density estimation. In *Proceedings of the Ninth IEEE International Conference on Computer Vision*, pages 664–671. IEEE, 2003.